2. Solving problems by search

13oct

Solving problems by search

In cazul in care scopul nu poate fi atins printr-o singura actiune, cum poate un agent gasi o **secventa de actiuni** care atinge scopul?

Reflex agent.
Goal-based agent:

Solving problems by search

In cazul in care scopul nu poate fi atins printr-o singura actiune, cum poate un agent gasi o **secventa de actiuni** care atinge scopul?

Reflex agent.
Goal-based agent: problem-solving agents - utilizeaza reprezentari atomice - structura interna a starii nu este vizibila
planning agents - goal-based agents care utilizeaza reprezentare structurata

Cuprins

- Agenti care rezolva probleme (Problem-solving Agents) - goal-based agents
- Formularea problemei
- Exemple
- Cautarea solutiilor
- Strategii de cautare neinformata - uninformed search strategies
- Strategii de cautare informata - informed search strategies

Problem-solving agents - Romanian problem (vezi 3.1 footnote)

performance measure:



- sa invat limba romana?
- admir peisaje
- sa ascult topul Billboard
- sa cunosc cat mai multi oameni
- sa ajung in Bucuresti - agentul are bilet de avion

Agenti care rezolva probleme (Problem-solving agent)

- Formularea **scopului** (Goal formulation)
  - Scopul = un set de stari ale lumii
- Formularea **problemei** = setul de actiuni si stari de luat in considerare
  - muta piciorul stang 10cm
  - invarte volanul 10 grade catre stanga
  - du-te de la Arad la Timisoara
- Exemplu:
  - Scop: du-te la Bucuresti
  - Actiuni: deplasarea dintr-un oras mare in altul
  - Arad: Timisoara, Sibiu, Zerind
  - care actiune e mai buna? niciuna nu e scopul

Problem-solving agents

Un agent cu
mai multe optiuni imediate de valori **necunoscute**
poate decide ce sa faca
daca **analizeaza** diferite **secvente posibile** care duc la o **stare cu valoare cunoscuta**.

SEARCH

Pt medii observabile, cunoscute (pt fiecare actiune se cunoaste starea in care se ajunge), discrete, deterministe (fiecare actiune are un singur rezultat) $\rightarrow$ solutia $=$ secventa de actiuni.

```
function SIMPLE-PROBLEM-SOLVING_AGENT(percept)
    returns action
 persistent: seq, an action sequence, initially empty
             state, some description of the current
                world state
             goal, a goal, initially null
             problem, a problem formulation
state← UPDATE-STATE(state, percep)
if seq is empty then
  goal←FORMULATE-GOAL(state)
  problem←FORMULATE-PROBLEM(state, goal)
  seq←SEARCH(problem)
  if seq=failure then return a null action
action ← FIRST(seq)
seq←REST(seq)
return action
```

Formulate – Search – Execute

Rezolvarea open loop - solutia este executata “cu ochii inchisi”.

Rezolvarea closed loop - actiuni in absenta unei cunoasteri

Formularea problemei

- starea initiala ex. "la Arad" $In(Arad)$
- actiuni aplicabile intr-o stare s
  $Go(Sibiu), Go(Timisoara), Go(Zerind)$
- model de tranzitie: functia $Result(s, a)$ starea care rezulta din realizarea actiunii a in starea s e.g.,
  $Result(In(Arad), Go(Zerind)) = In(Zerind)$
  succesor = stare ce poate fi atinsa dintr-o stare printr-o singura actiune
  - spatiul starilor (state space): un graf definit de starea initiala, actiuni, model de tranzitie
  - cale (path): o secventa de stari conectate printr-o secventa de actiuni
- test al scopului (goal test) , poate fi
  - explicit, e.g., $x =$ "la Bucuresti"
  - implicit, e.g., $NoDirt(x)$
- costul (path cost) (aditiv) - ex. suma distantelor, numarul de actiuni executate
  $c(s, a, s')$ este costul unui pas, presupus a fi ≥ 0

Formularea problemei - cont.

O solutie este o secventa de actiuni care conduc de la starea initiala la starea scop.
O solutie optima este o solutie care are cel mai mic cost.

Problemele de cautare sunt modele



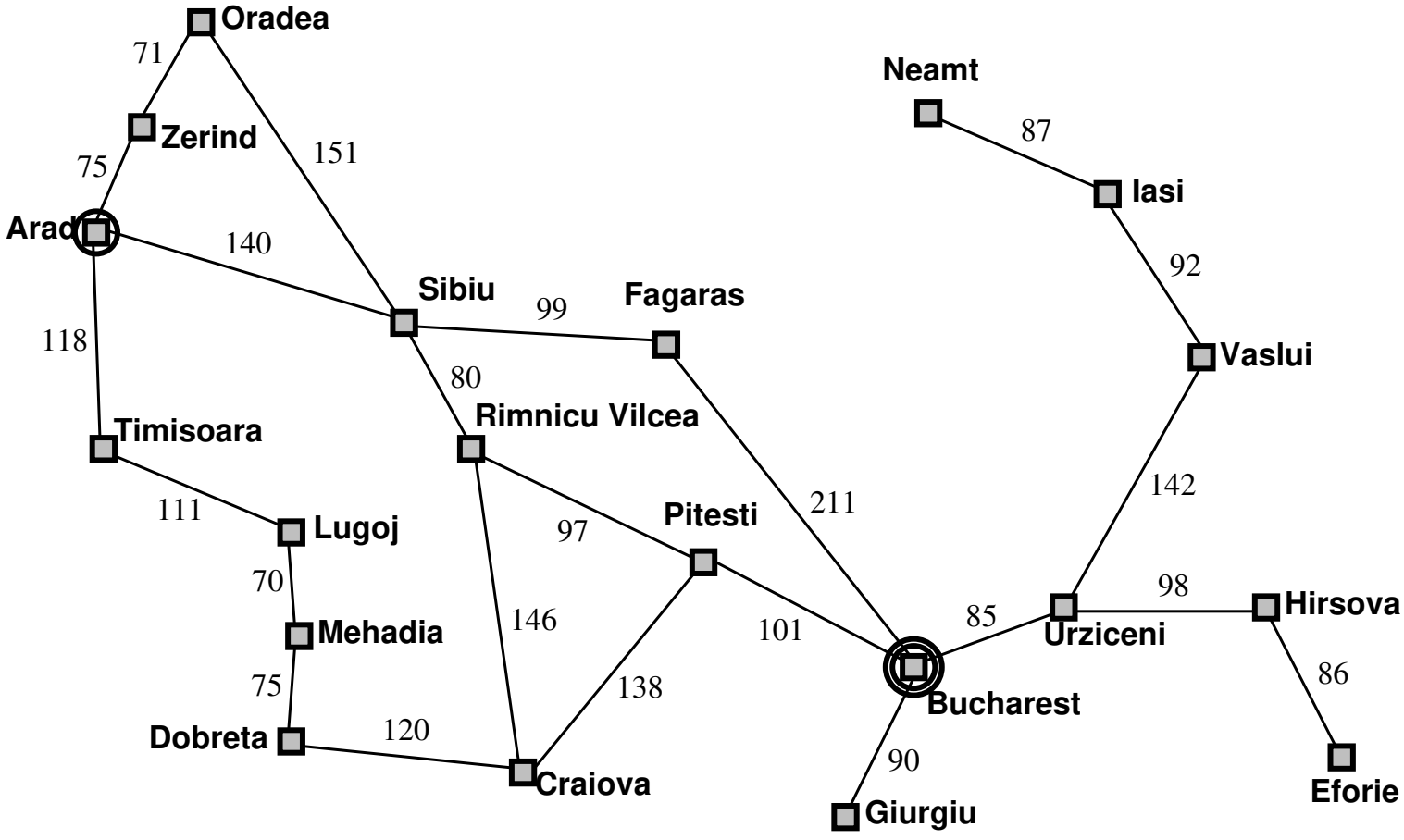
Abstractizare - eliminarea detaliilor din reprezentare

Abstractizare

Lumea reala este extrem de complexa $\Rightarrow$ spatiul starilor trebuie **abstractizat**

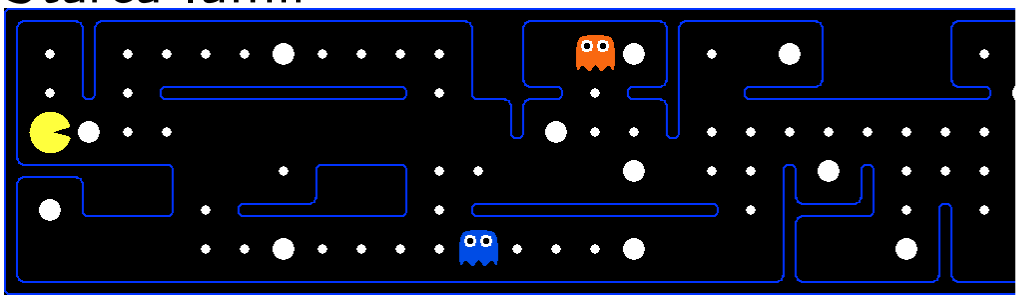
- stare (Abstracta) = un set de stari reale
- actiune (Abstracta) = o combinatie complexa de actiuni reale
  ex., "Arad $\rightarrow$ Zerind" reprezinta un set de rute posibile, opriri, posturi de radio ascultate
- solutie (Abstracta) = un set de cai reale care sunt solutie in lumea reala
  Fiecare actiune abstracta trebuie sa fie mai simpla decat actiunea reala

Romania


Oradea
71
Neamt
Zerind
87
151
75
Iasi
Arad
140
92
Sibiu
99
Fagaras
118
Vaslui
80
Rimnicu Vilcea
Timisoara
142
111
211
Pitesti
Lugoj
97
98
70
Hirsova
146
85
101
Mehadia
Urziceni
75
138
86
Bucharest
Dobreta
120
90
Craiova
Eforie
Giurgiu

Pacman - alegerea starilor

- Starea lumii



- Starea dupa abstractizare

Gaseste o anumita pozitie

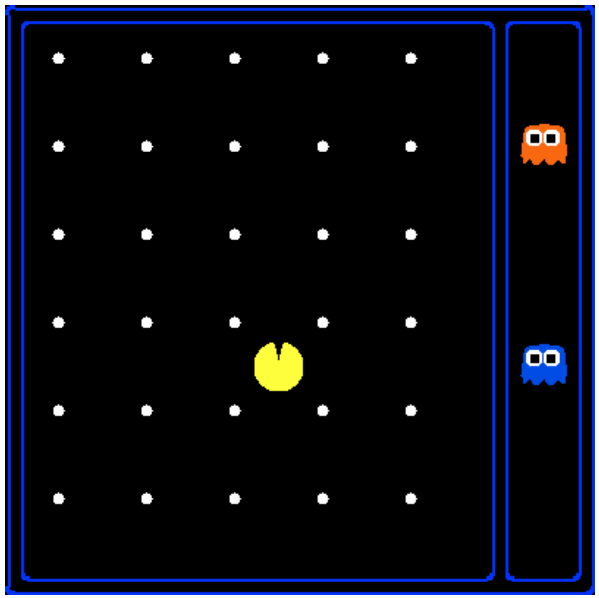
- stari: (x,y)
- actiuni: NSEW
- succesor: actualizarea pozitiei
- goal test: (x,y)=End

Mananca toate punctele

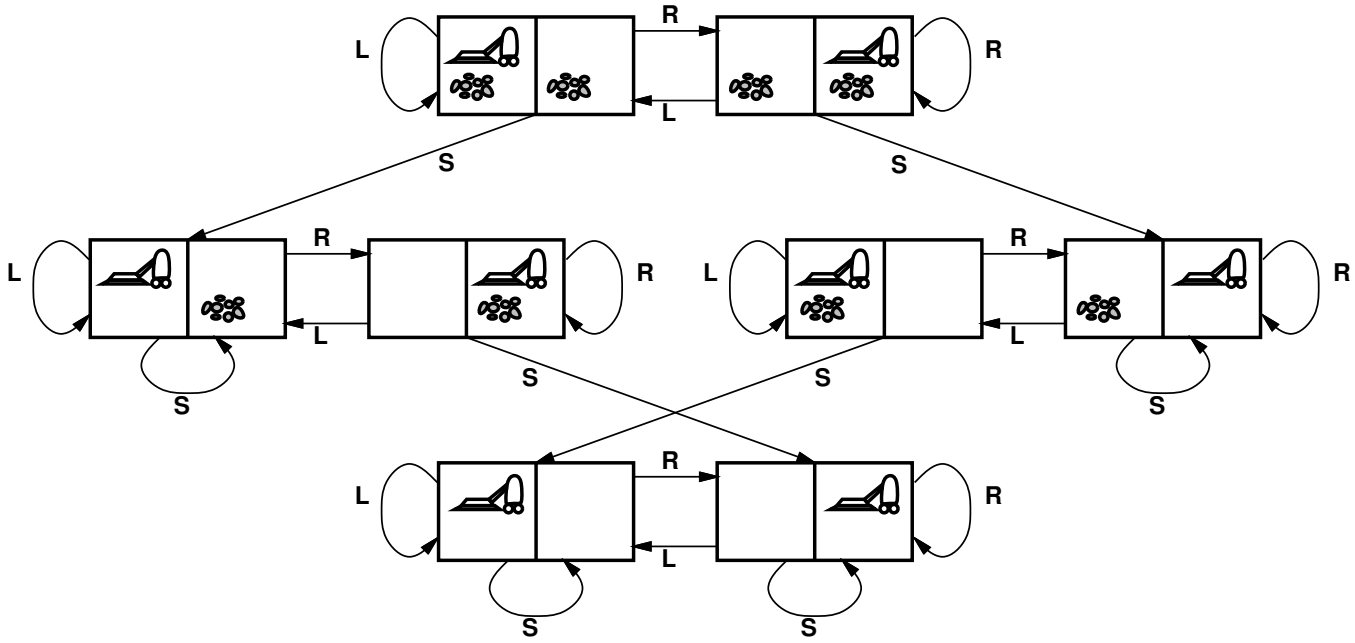
- stari: (x,y), boolean pt mancare
- actiuni: NSEW
- succesor: actualizarea pozitiei si a mancarii
- goal test: toate valorile asociate mancarii sunt false

Pacman - numarul starilor

- Starea lumii
  - pozitia agentului: 120
  - bucati de mancare: 30
  - pozitiile fantomelor: 12
  - orientarea agentului: NSEW
- Numar de stari
  - stari ale lumii: $120x(2^{30})x(12^2)x4$
  - stari pentru gasirea unui punct: 120
  - stari pentru a mananca toate punctele: $120x(2^{30})$

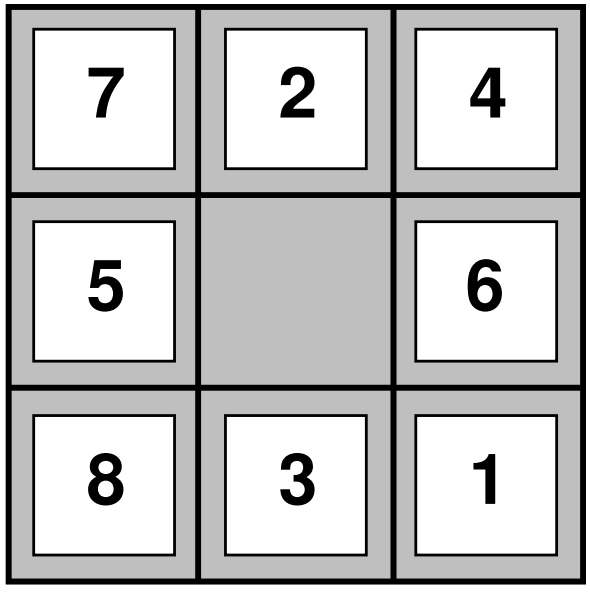


Toy problem: lumea aspiratorului

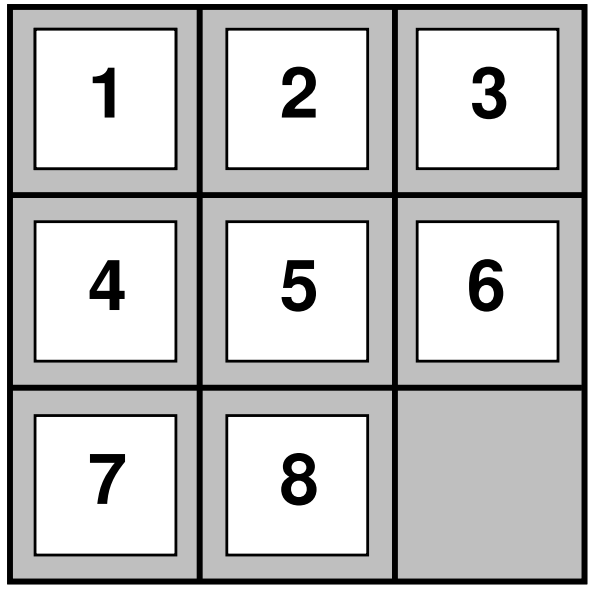


- stari: pozitia robotului si localizarea mizeriei: $2x2^2$ $(nx2^n)$
- starea initiala: oricare
- actiuni: *Left*, *Right*, *Suck*
- modelul de tranzitie
- goal test: toate camerele sa fie curate
- cost: 1 per actiune, numarul de pasi din cale

Toy problem: 8-puzzle



Start State



Goal State

- states: pozitia pieselor (ignora pozitiile intermediare)
- starea initiala:
- actions: muta spatiul liber la stanga, dreapta, sus, jos
- modelul de tranzitie
- goal test
- path cost - 1 per mutare

Stari care pot fi atinse: $9!/2$

Real-world problem

Airine travel problem

- states: locatie, timp curent, segmente anterioare, tip de zbor: intern/interantional
- starea initiala: specificata de utilizator
- actions: zbor de la locatia curenta, in ore clasa, cu asigurarea timpului necesar de transfer
- modelul de tranzitie
- goal test: s-a ajuns la destinatie?
- path cost: pret, timp de asteptare, proceduri de emigrare, calitatea locului, tipul aeronavei, discounturi pt client,..

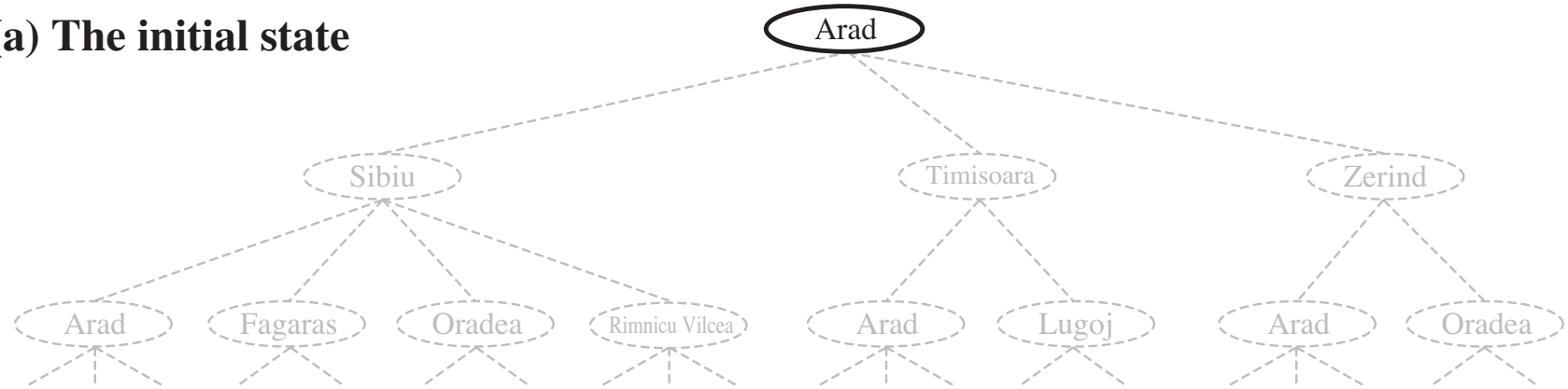
Asamblare automata. Designul proteinelor

3. Solutionarea problemelor de cautare

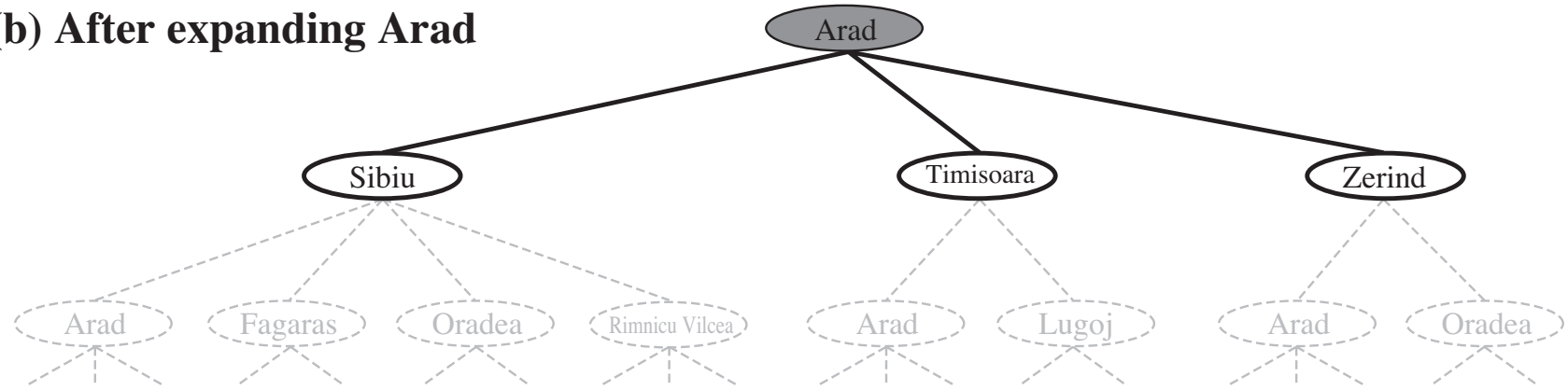
- considerarea secventelor posibile de actiuni
- evitarea construirii intregului graf al spatiului starilor $\rightarrow$ arborele pt "what if" arbore de cautare (search tree)
- urmarea unei optiuni acum si pastrarea altora pentru cazul in care prima alegere nu conduce catre o solutie
- search tree: radacina, noduri, expandare, nod parinte, nod copil, nod frunza, **frontiera** (open list)

Cautare cu arbore de cautare

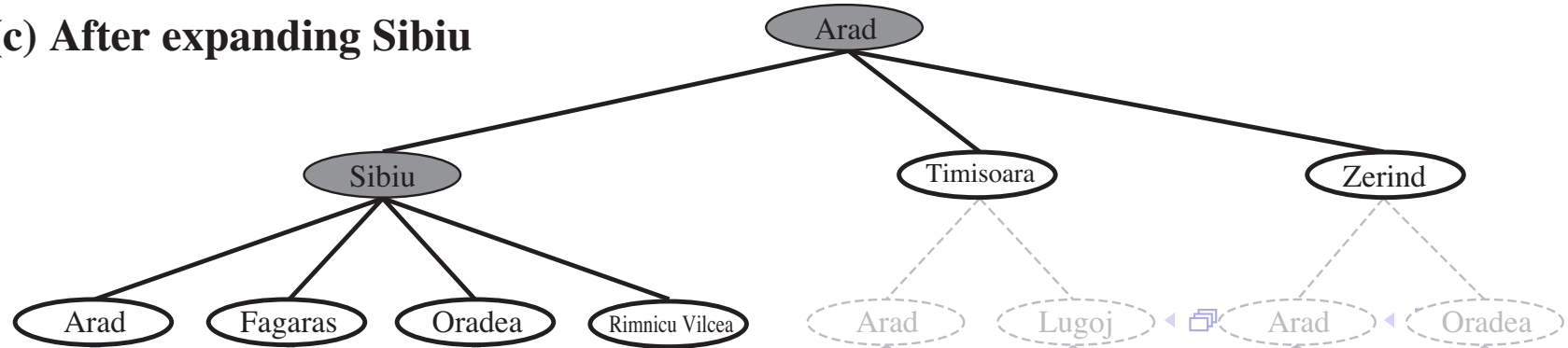
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



https://www.movingai.com/SAS/ASM/

Tree Search - Graph Search

```
function TREE-SEARCH(problem) returns a solution, or
    failure
initialize the frontier using the initial state of
    problem

 loop do
   if the frontier is empty then return failure
   choose a leaf node and remove it from the frontier
   if the node contains a goal state
       then return the corresponding solution
   expand the chosen node, adding the resulting nodes
        to the frontier
```

Tree Search - Graph Search

```
function TREE-SEARCH(problem) returns a solution, or
    failure
initialize the frontier using the initial state of
    problem

 loop do
   if the frontier is empty then return failure
   choose a leaf node and remove it from the frontier
   if the node contains a goal state
       then return the corresponding solution
   expand the chosen node, adding the resulting nodes
        to the frontier
```

frontiera - open list

Loopy path (Redundant path): Arad - Sibiu - Arad: $\rightarrow$ explored set (closed list)

Tree Search - Graph Search

```
function GRAPH-SEARCH(problem) returns a solution, or
    failure
 initialize the frontier using the initial state of
    problem
 initialize the explored set to be empty

loop do
  if the frontier is empty then return failure
  choose a leaf node and remove it from the frontier
  if the node contains a goal state then return the
     corresponding solution
  add the node to the explored set
  expand the chosen node, adding the resulting nodes
     to the frontier
    only if not in the frontier or explored set
```

Arbori de cautare

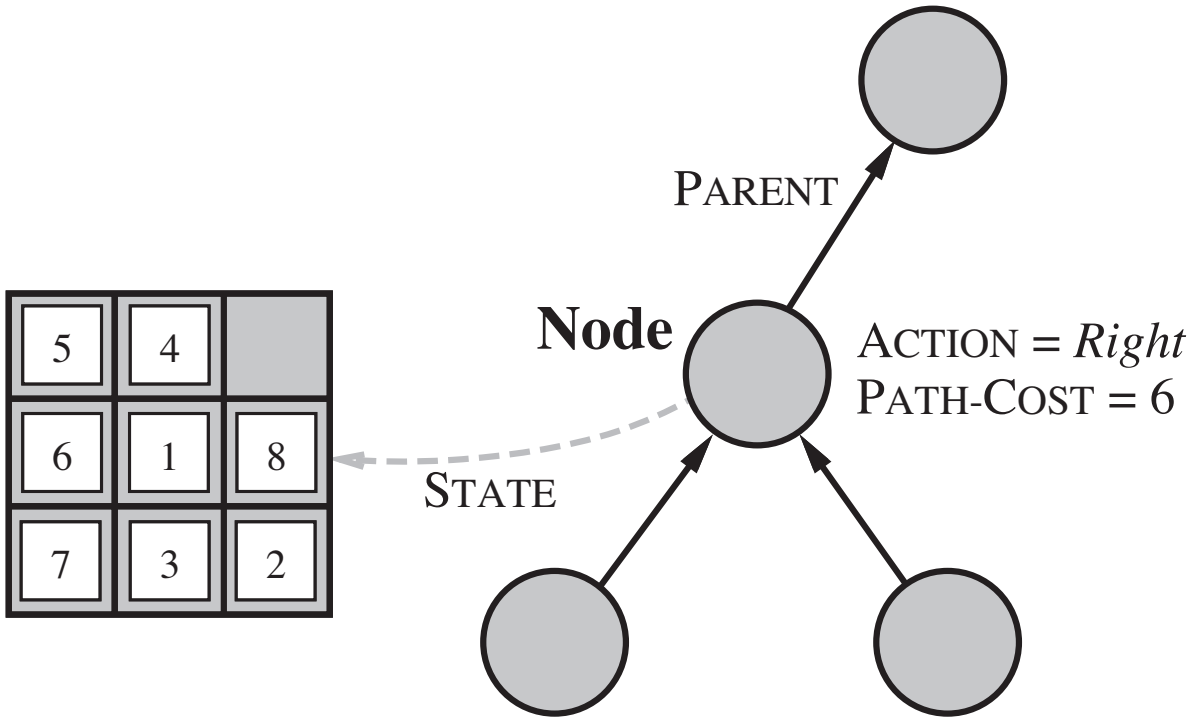
- expandarea planurilor potentiale (nodurile din arbore)
- mentinerea unei liste de noduri de frontiera - separa graful spatiului de cautare in regiune explorata si regiune neexplorata
- se incearca expandarea a cat mai putine noduri
  Obs:pentru cele mai multe probleme, nu putem construi intreg arborele

Secventa de arbori de cautare generati cu cautare in graf:



Infrastructura pentru algoritmi de cautare

- reamintire: o stare este o reprezentare a configuratiei fizice
- Un nod din arbore de cautare:
  - starea
  - parintele - nodul din arbore care a generat acest nod
  - actiunea - actiunea care a fost aplicata parintelui pentru generarea nodului
  - costul caii - g(n) de la nodul radacina



Implementare

frontiera = structura ce pastreaza toate nodurile care mai trebuie expandate

- FIFO queue: coada - scoate cel mai vechi element din lista
- LIFO queue: stiva - scoate cel mai recent element
- priority queue: lista de prioritati - scoate un element cu cea mai mare prioritate conform unei functii de ordonare

explored set - poate fi implementat ca o tabela de hashing

Masurarea performantei

- strategie - alegerea unei **ordini de expandare a nodurilor**
- dimensiunile pentru evaluarea strategiilor:
  - completeness —gaseste intotdeauna o solutie daca aceasta exista
  - optimality—gaseste intotdeauna solutia cu costul cel mai mic
  - time complexity —cat dureaza gasirea unei solutii (numar de noduri expandate)
  - space complexity—cata memorie este necesare

Complexitatea spatiului si a timpului:

- b—factor de ramificare (branching factor) - numar maxim de succesori pentru un nod
- d—adancimea (depth) solutiei celei mai putin adanci
- m—lungimea maxima a unui path in state space

Recapitulare

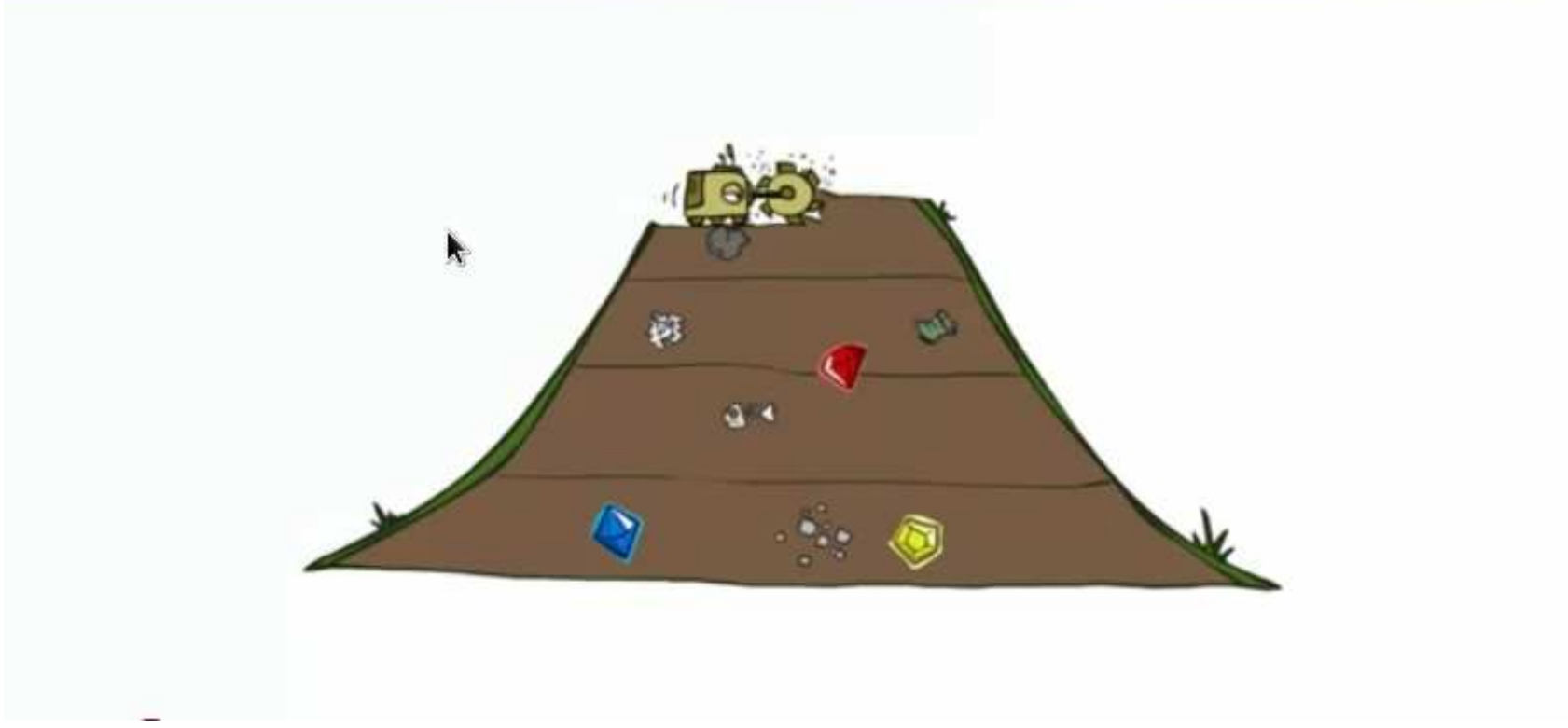
- Search problem
  - stari (configuratii ale lumii)
  - actiuni si cost
  - modelul de tranzitie (succesori) - dinamica lumii
  - starea de start si testul de scop (goal test)
- Search tree (arborele de cautare)
  - noduri - reprezinta planuri pt a atinge starile
  - planurile au cost
- Search algorithm
  - construiesc sistematic arborele de cautare - cat mai putin
  - aleg ordinea nodurilor in frontiera
  - optimal: solutia de cost minimc

4. Strategii de cautare neinformata

Strategiile neinformate - numai informatie disponibila din definirea problemei

- Cautarea in latime (Breadth-first search)
- Cautarea cu cost uniform (Uniform-cost search)
- Cautarea in adancime (Depth-first search)
- Cautarea cu adancime limitata (Depth-limited search)
- Cautarea cu adancime incrementala (Iterative deepening search)

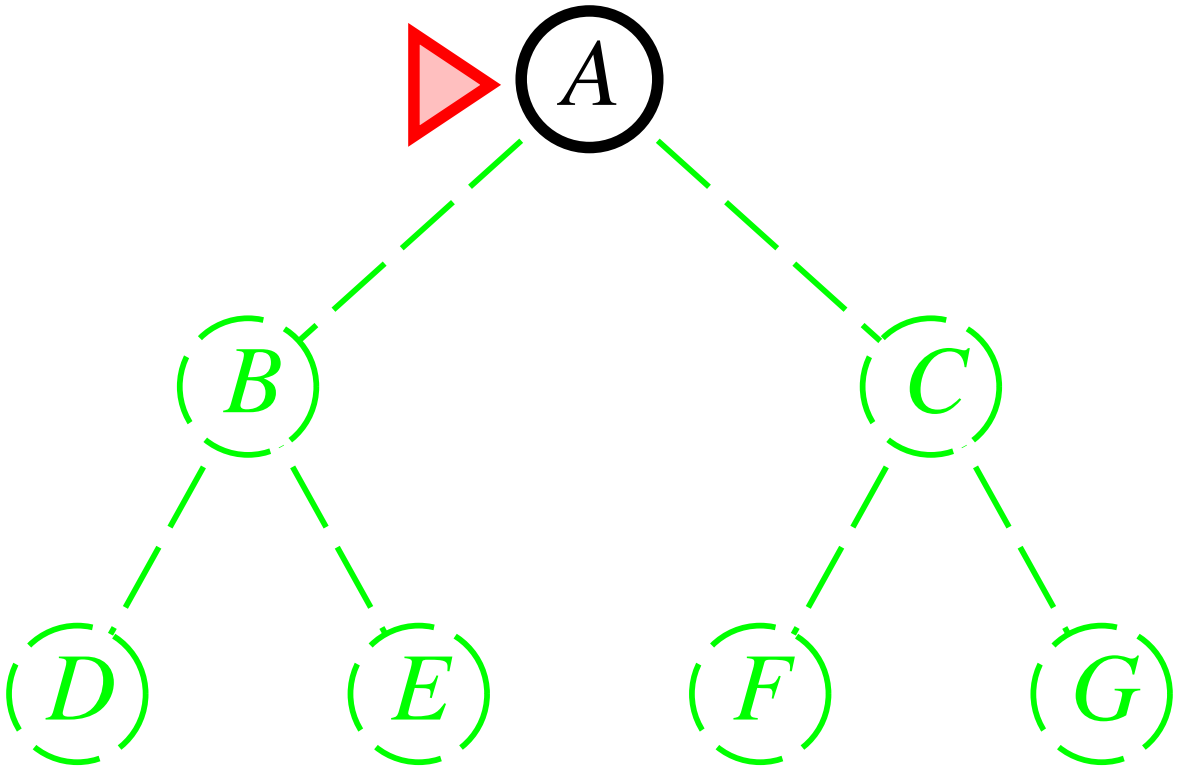
Breadth-First Search



Imagine preluata din curs AI edXedge

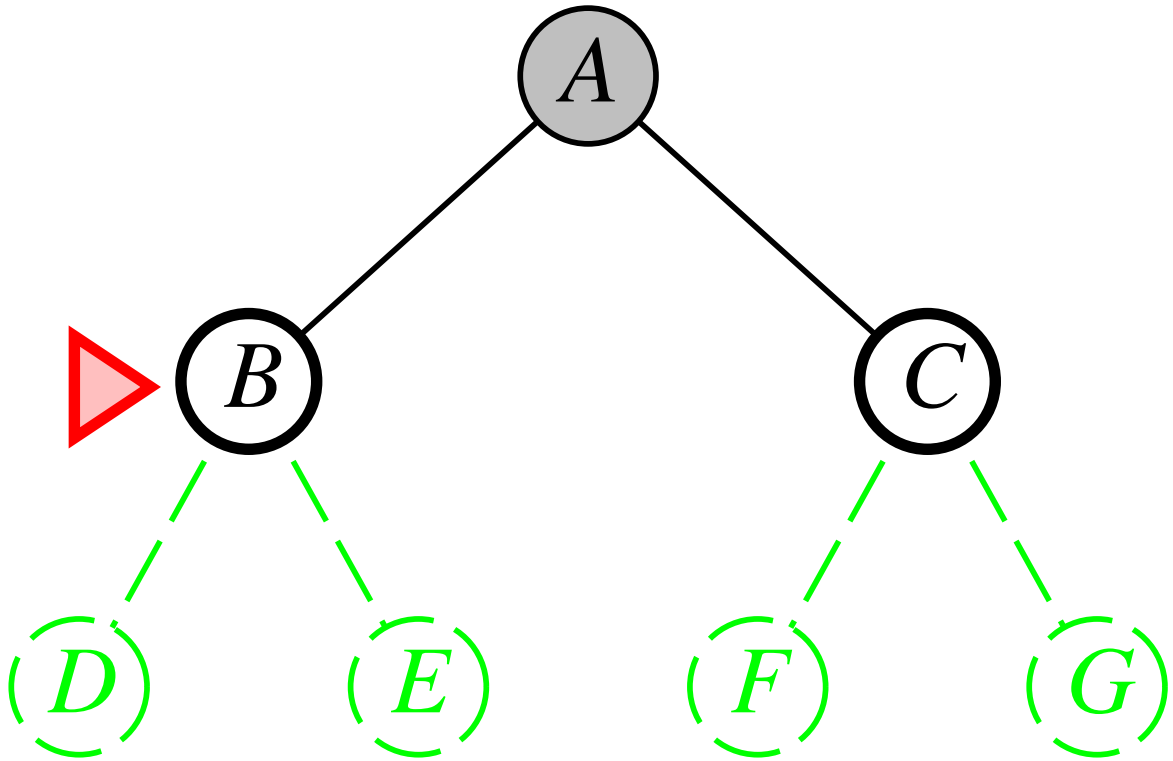
Cautarea in latime

- Expandeaza cel mai superficial nod neexpandat
- frontiera - FIFO queue: succesorii noi sunt adaugati la final



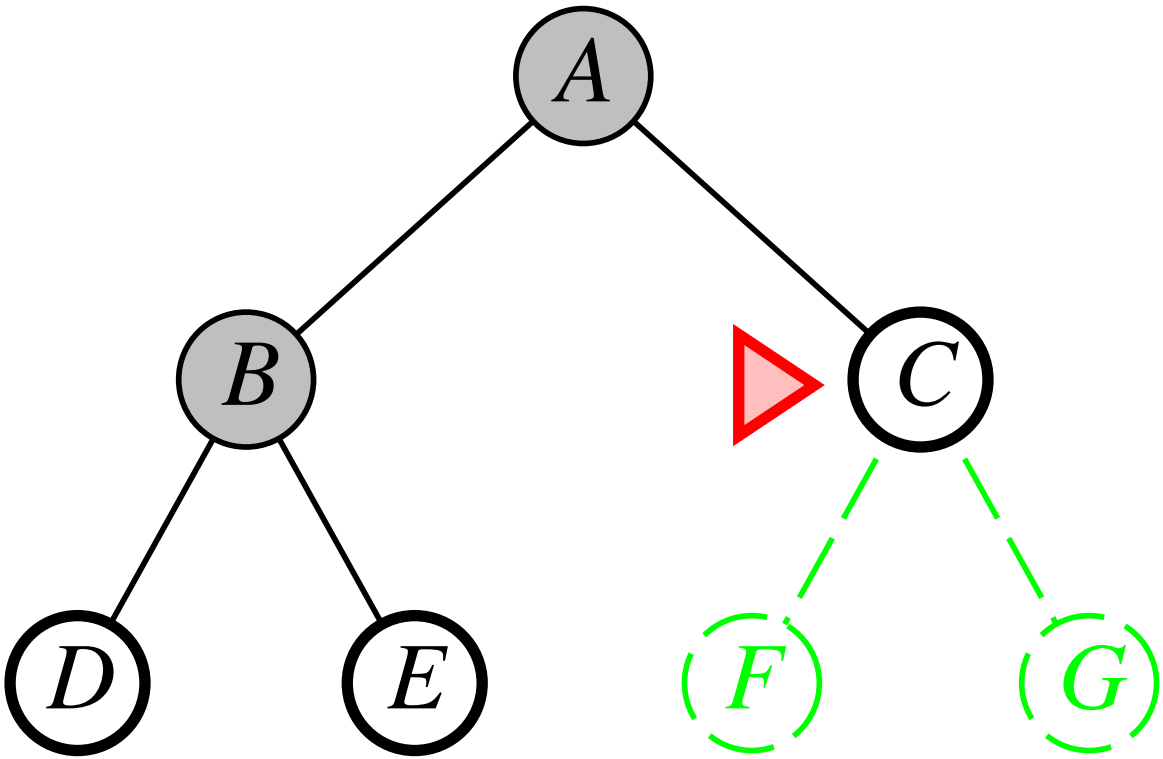
Cautarea in latime

- Expandeaza cel mai superficial nod neexpandat
- frontiera - FIFO queue: succesorii noi sunt adaugati la final



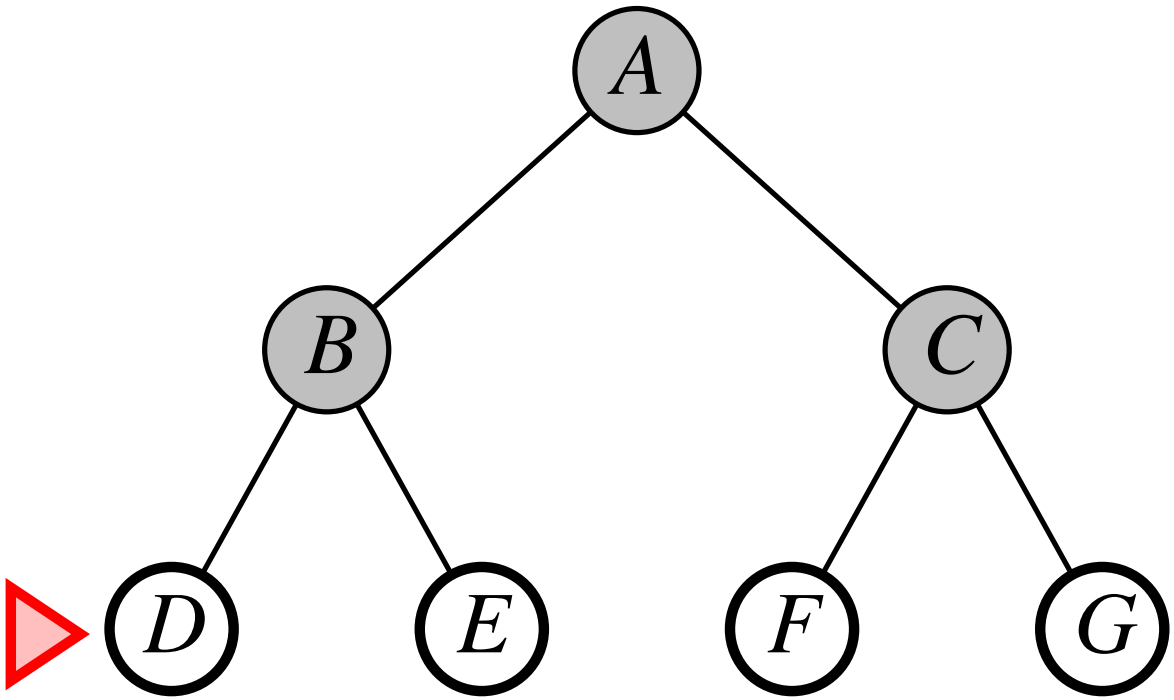
Cautarea in latime

- Expandeaza cel mai superficial nod neexpandat
- frontiera - FIFO queue: succesorii noi sunt adaugati la final



Cautarea in latime

- Expandeaza cel mai superficial nod neexpandat
- frontiera - FIFO queue: succesorii noi sunt adaugati la final



Breadth-first search - caz particular al graph-search

Graph-search algorithm with FIFO pt frontiera +
!!! testul de scop este aplicat la momentul generarii unui nod, nu la momentul scoaterii lui din frontiera

Proprietatile cautarii in latime

- Complet?: DA (daca b este finit)
- Timp: $1 + b + b^2 + b^3 + \ldots + b^d = O(b^{d+1})$, i.e., exp. in d
- Spatiu: $O(b^d)$ (pastreaza toate nodurile in memorie: $O(b^{d-1})$ explored set + $O(b^d)$ frontiera)
- Optimal: DA (daca cost $= 1$ per step);

Spatiul este o foarte mare problema

Time and memory requirements for bredth-first search

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

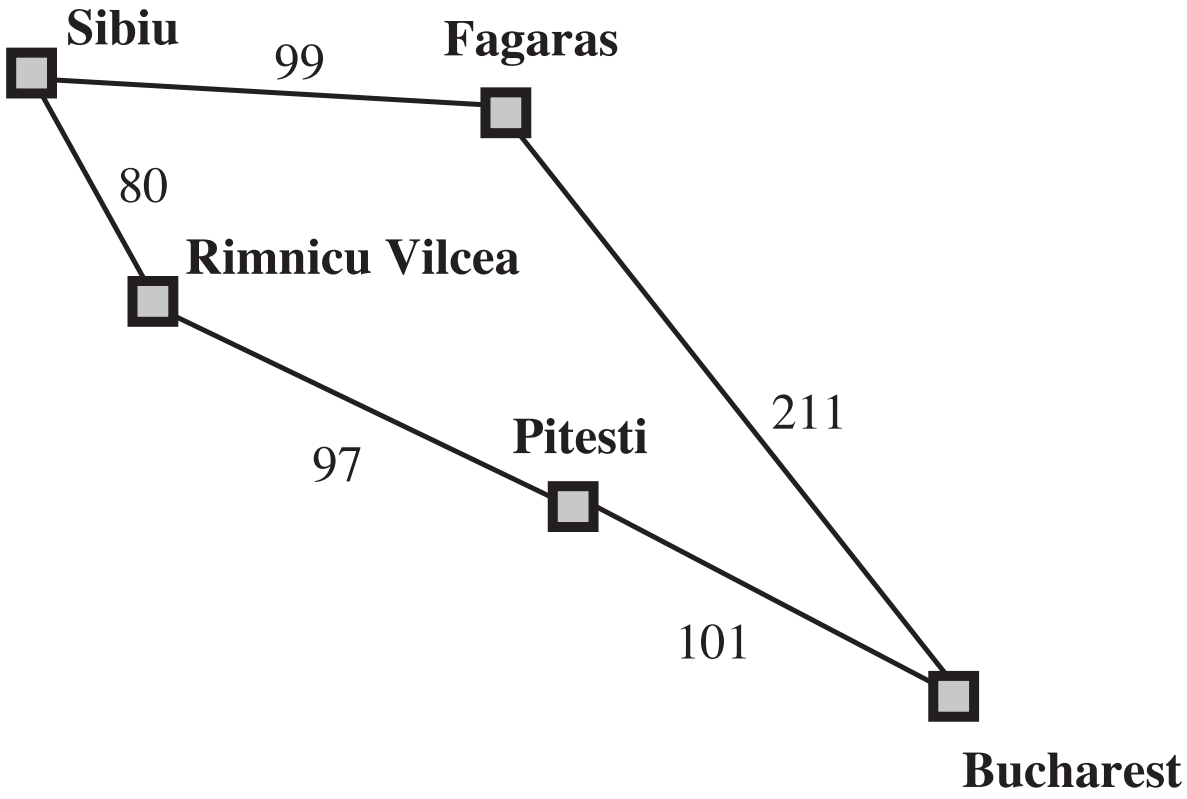
b=10, 1 million nodes/second; 1000bytes/node

Cautarea cu cost uniform (Uniform-cost search)

- Expandeaza nodul cu cel mai mic cost
- frontiera = coada ordonata dupa costul caii; nodul cu g cel mai mic este expandat primul

Echivalent cu cautarea in latime daca costul oricarui pas este acelasi

Cautarea cu cost uniform (Uniform-cost search)



Nod expandat	Noduri adaugate in frontiera
Sibiu	Rimnicu Valcea g=80, Fagaras g=99
Rimnicu Valcea	Pitesti g=80+97=177
Fagaras	Bucuresti g=99+211=310
Pitesti	Bucuresti g=80+97+101=278
Bucuresti	goal test returneaza true

Observatie: testul de goal se aplica doar la scoaterea din frontiera

Proprietati ale cautarii cu cost uniform

- completitudinea este garantata daca costul fiecarei actiuni depaseste 0; cu actiuni de cost 0 se ajunge la cai infinite
- este optimal
- d,b nu ghideaza cautarea, ci costul g: complexitatea spatiului si a timpului poate fi mai mare decat cea a cautarii in adancime

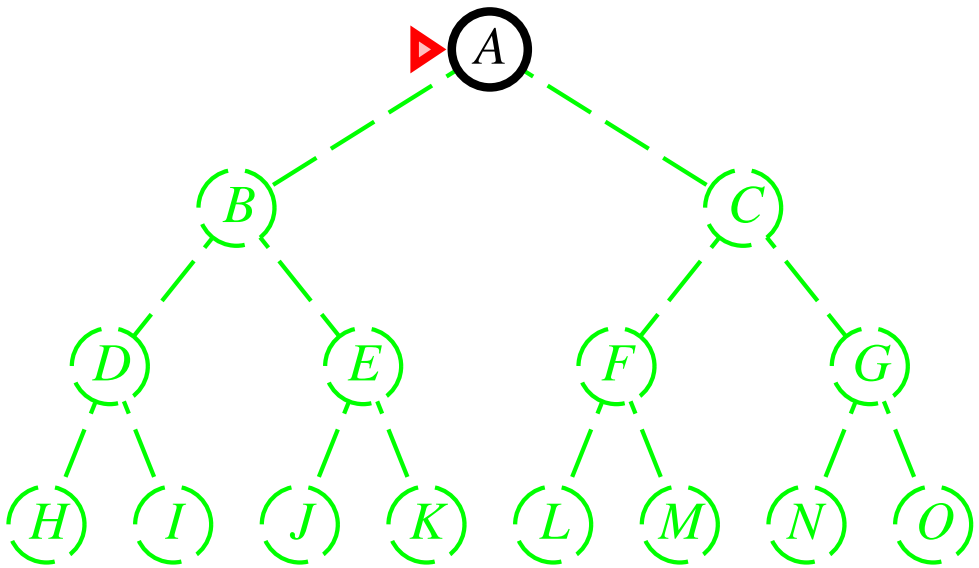
Depth-First Search



UC Berkeley via edX

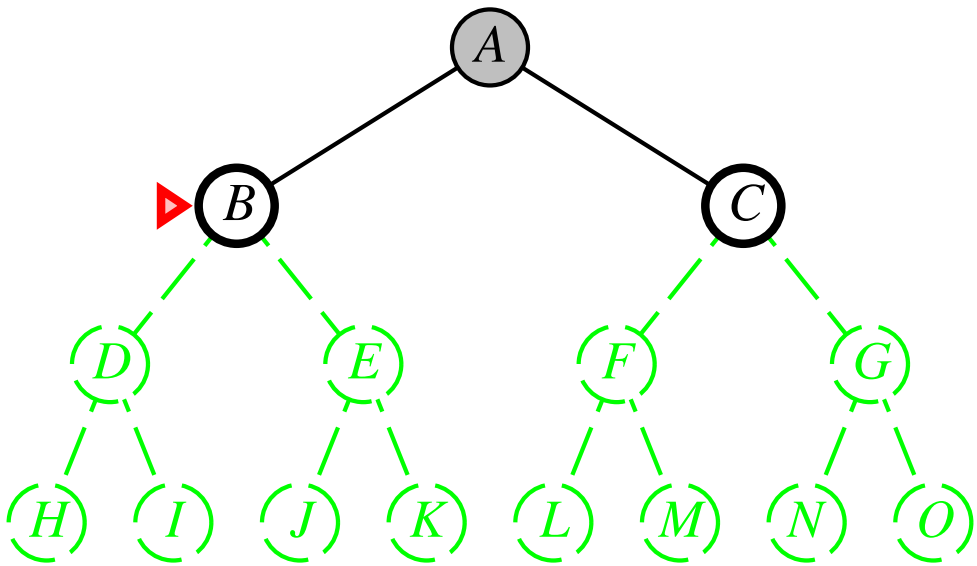
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



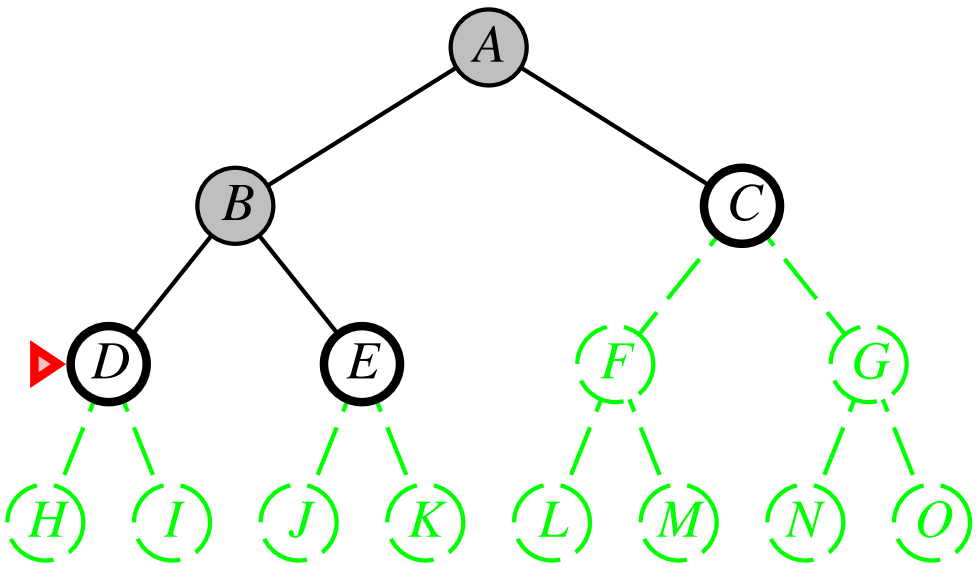
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



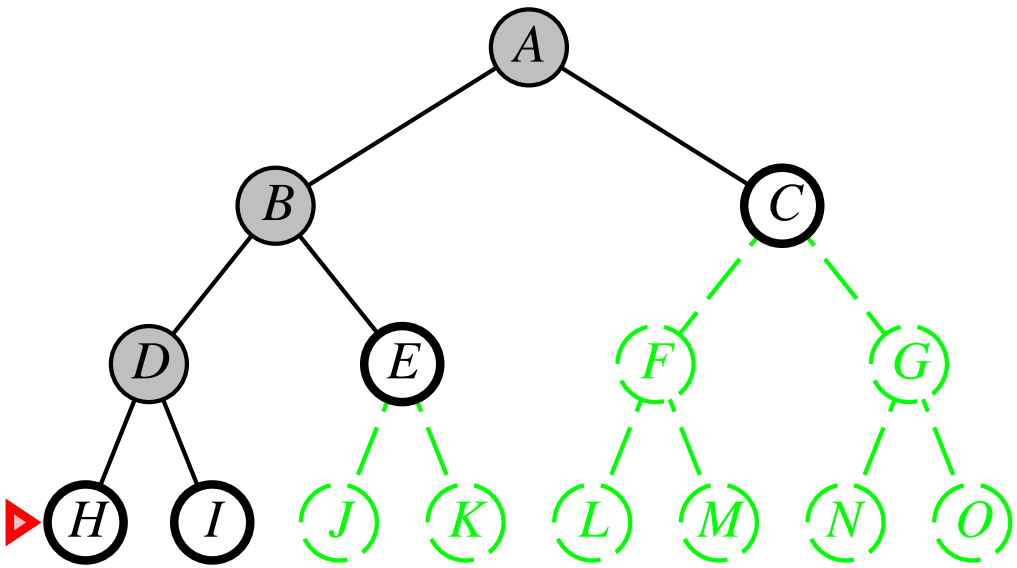
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



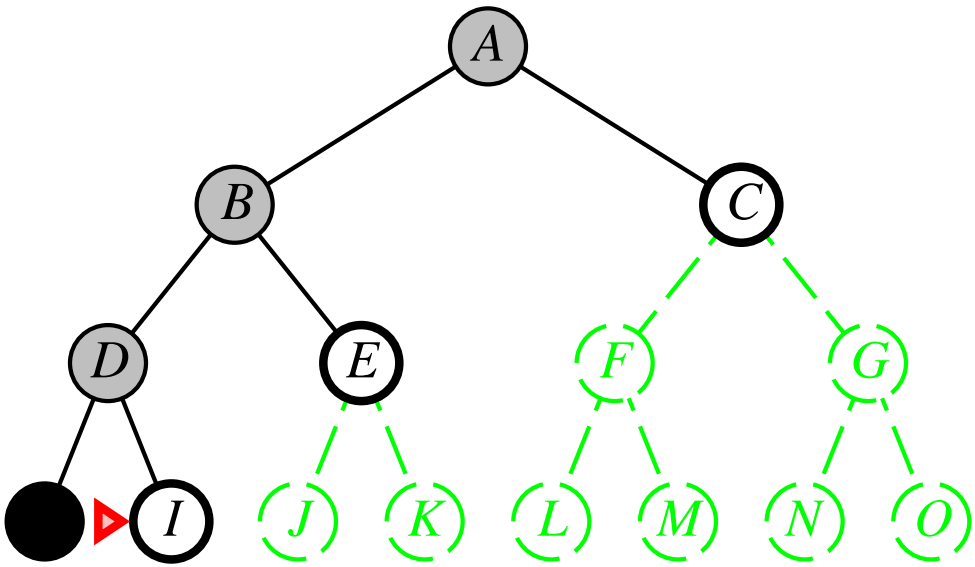
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



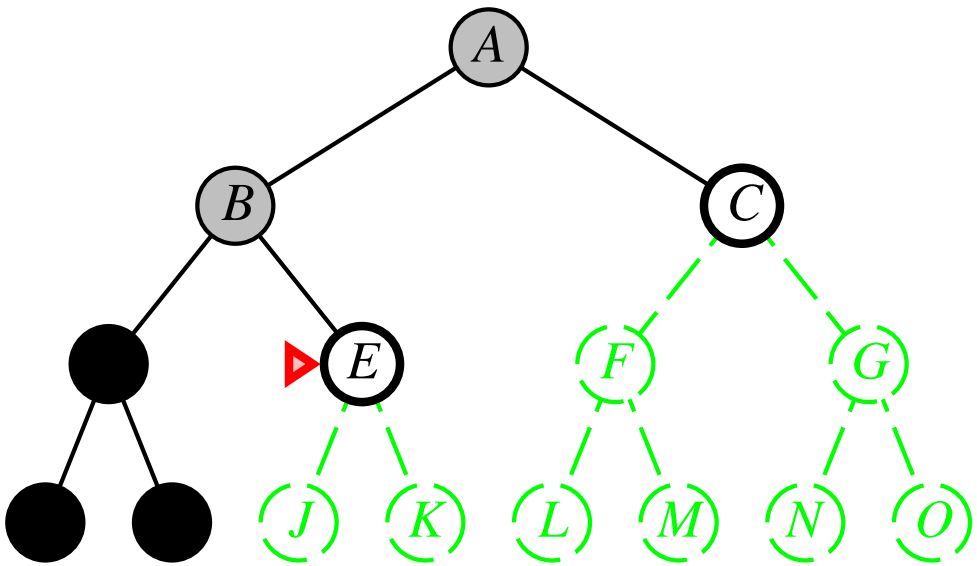
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



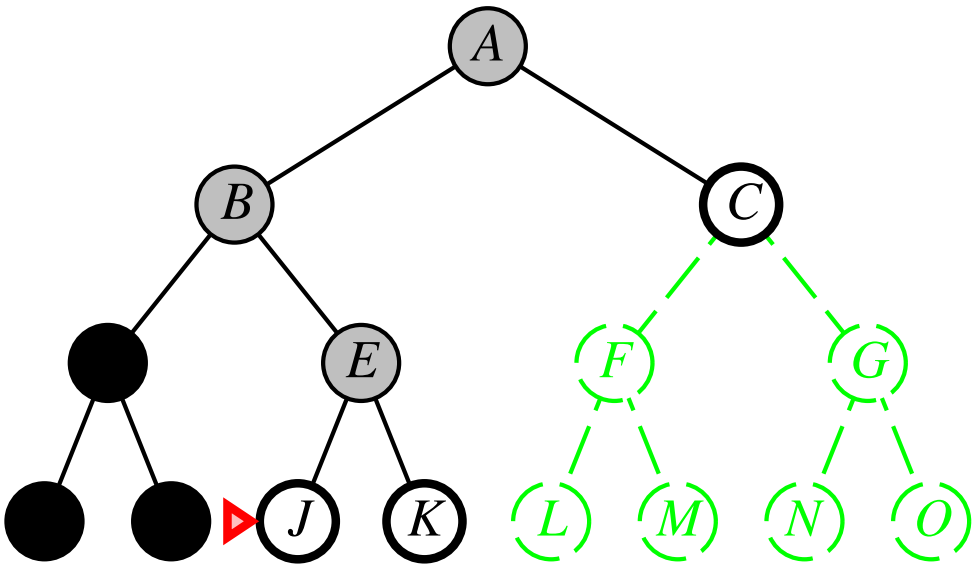
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



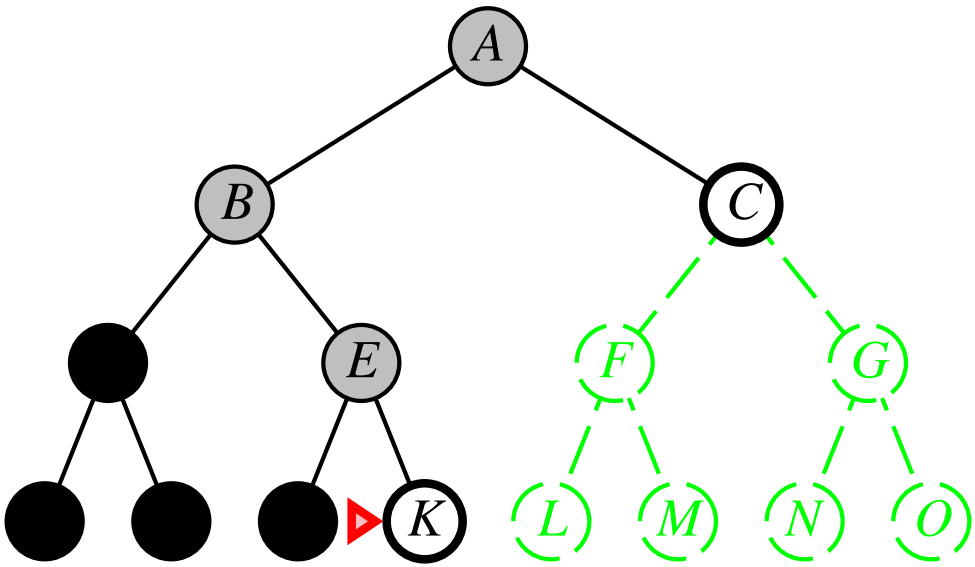
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



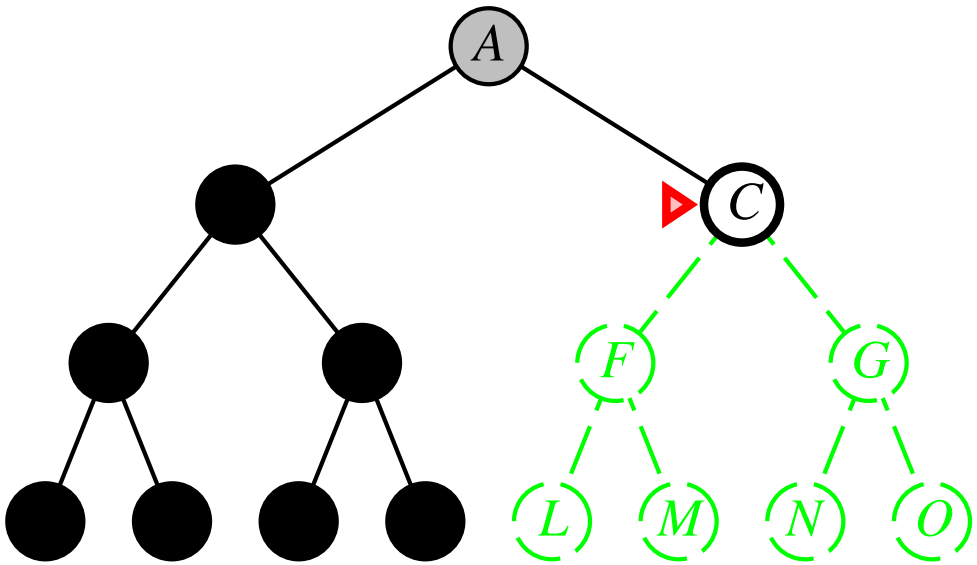
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



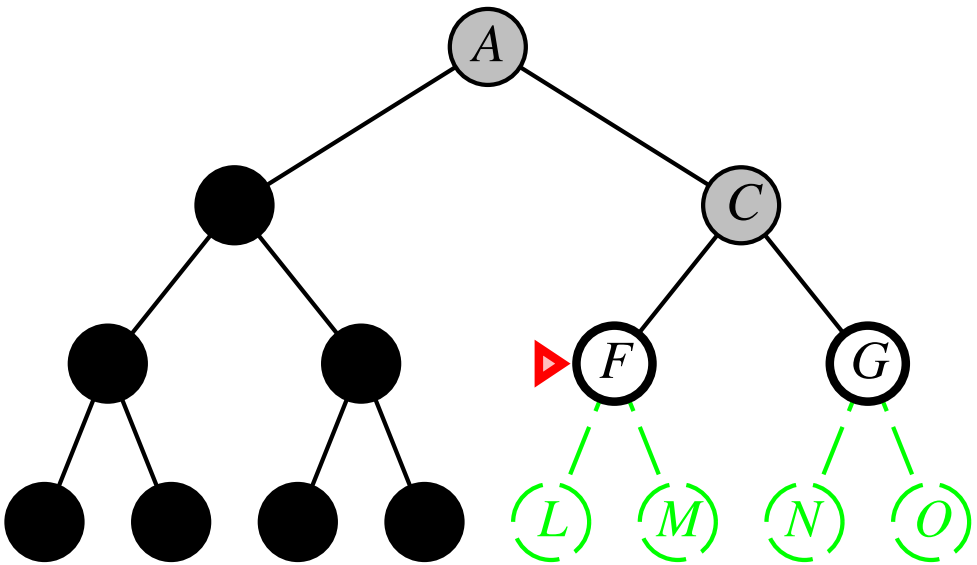
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



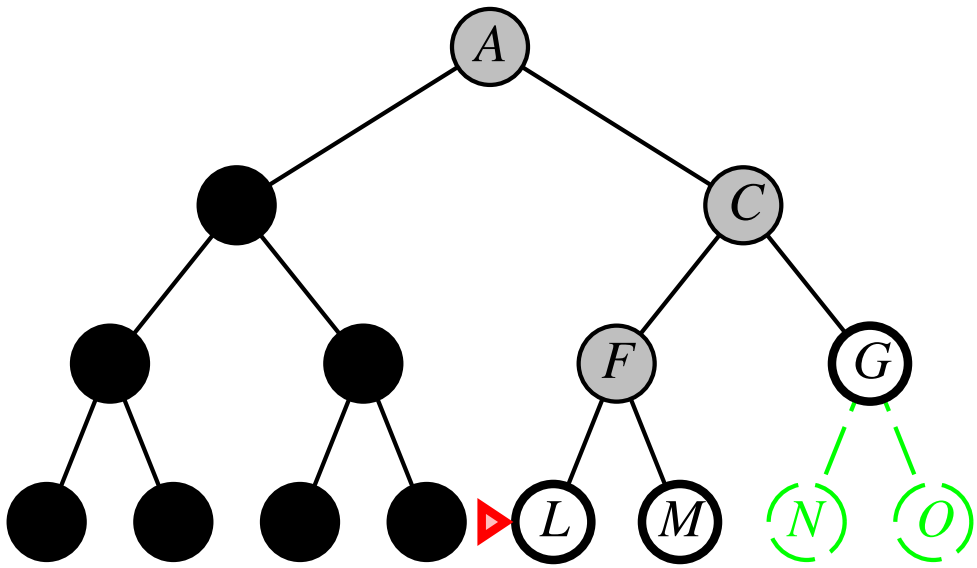
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



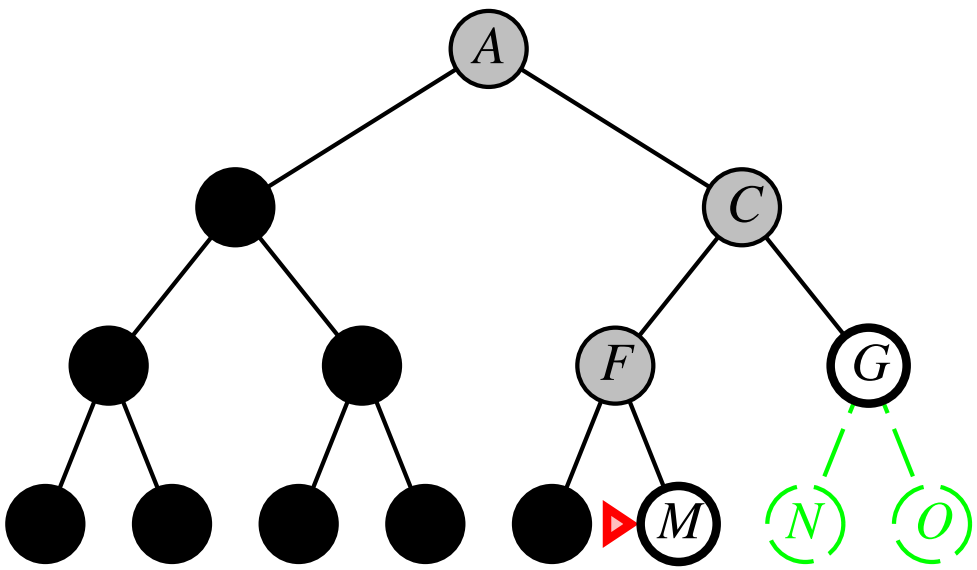
Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



Cautarea in adancime

- Expandeaza cel mai adanc nod neexpandat
- frontiera: LIFO (stiva), i.e., pune succesorii la inceput
- nodurile expandate fara succesori in frontiera sunt eliminate din memorie



Proprietatile cautarii in adancime

- complet?
  - graph search - DA, pentru spatii de stari finite
  - tree search - NU, datorita cailor redundante: Arad-Sibiu-Arad-Sibiu..
  - spatii de stari infinite: NU
- optim?: NU: ex, daca C si J sunt ambele scopuri, J va fi solutia, desi C e solutie mai buna
- timp: tree search $O(b^m)$: f. rau daca m - adancimea maxima a oricarui nod - este mult mai mare decat d
  daca solutiile sunt dense, poate fi mai rapid decat alg de cautare in latime
- spatiu: $O(bm)$, i.e., spatiu liniar !!!

Observatie: backtracking search - o varianta a cautarii in adancime: doar un succesor este expandat, nu toti O(m)

Cautarea cu adancime limitata (Depth-limited search)

- depth-first search cu adancime limitata la l nodurile la adancimea l nu au succesori
- modificare a graph-search/tree-search sau implementare recursiva

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns
    a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.
      INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns
   a solution, or failure/cutoff
 if problem.GOAL-TEST(node.STATE) then return
    SOLUTION(node)
 else if limit = 0 then return cutoff
 else
   cutoff_occurred? ← false
   for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      result ← RECURSIVE-DLS(child,problem,limit-1)
      if result = cutoff then cutoff_occurred? ← true
      else if result ≠ failure then return result
   if cutoff_occurred? then return cutoff else return
       failure
```

Cautare cu adancime incrementala (Iterative deepening search)

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns
    a solution, or failure

 for depth = 0 to ∞ do
     result ← DEPTH-LIMITED-SEARCH(problem, depth)
     if result ≠ cutoff then return result
```

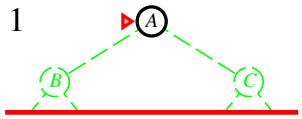
Iterative deepening search

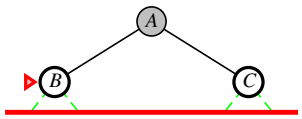
Limit = 0

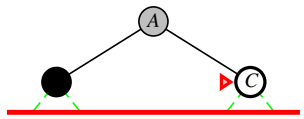


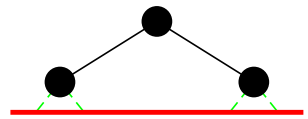
Iterative deepening search

Limit = 1



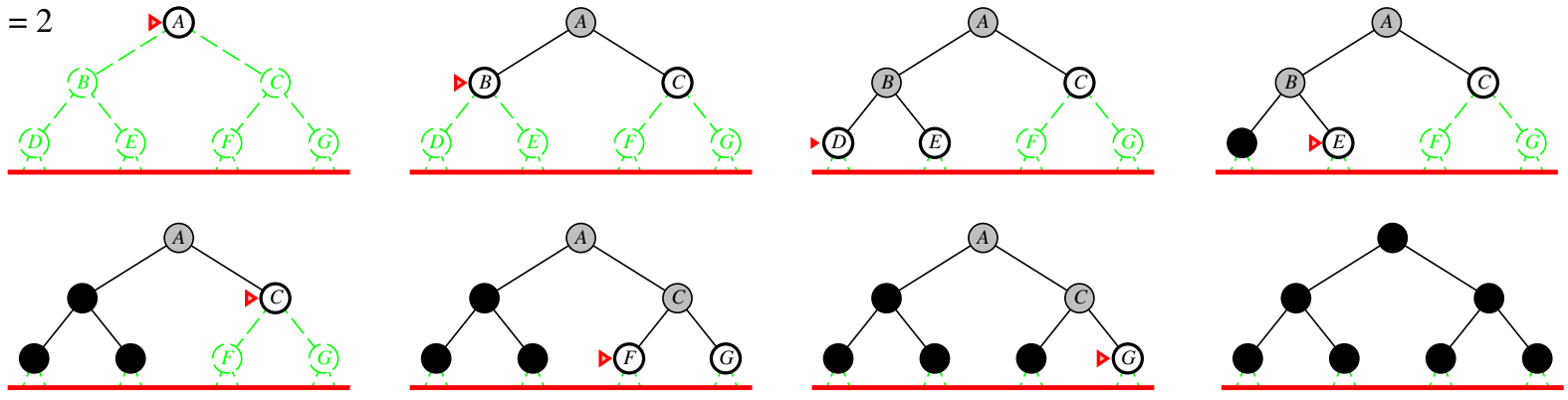




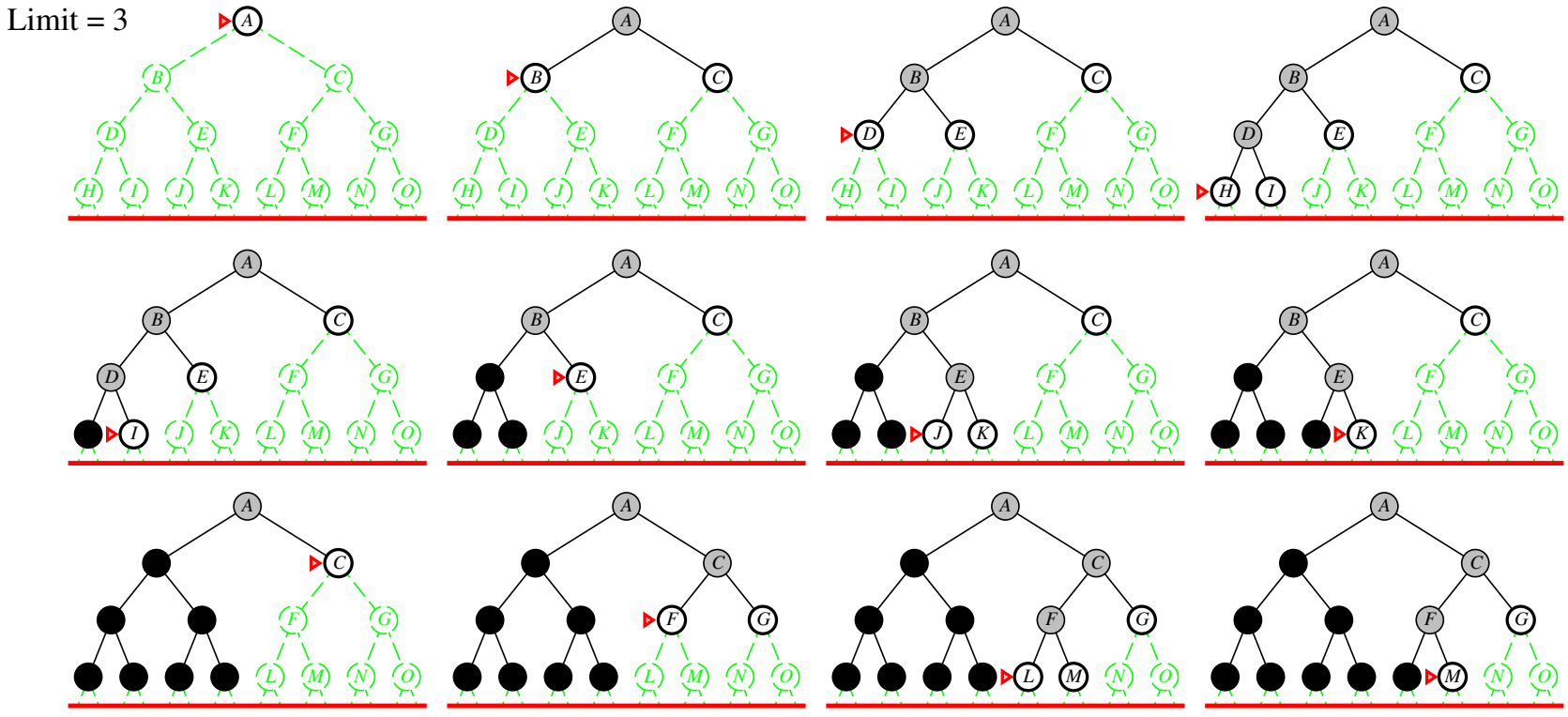


Iterative deepening search

Limit = 2



Iterative deepening search



Proprietati ale cautarii iterative deepening

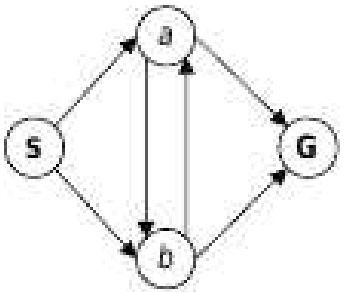
- Complet: DA
- Time: $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- Space: $O(bd)$
- Optimal: DA, daca toate actiunile au acelasi cost

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

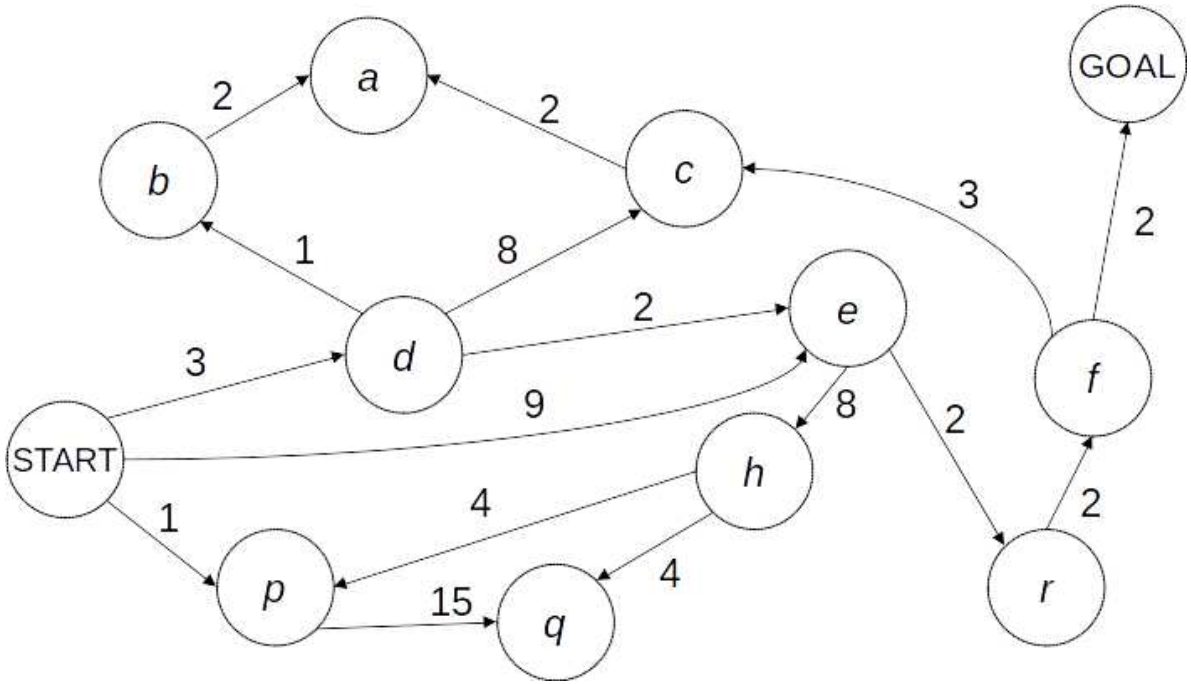
Quiz 1

Considerand graful de spatiu al starilor, spuneti cate noduri include arborele de cautare (search tree)



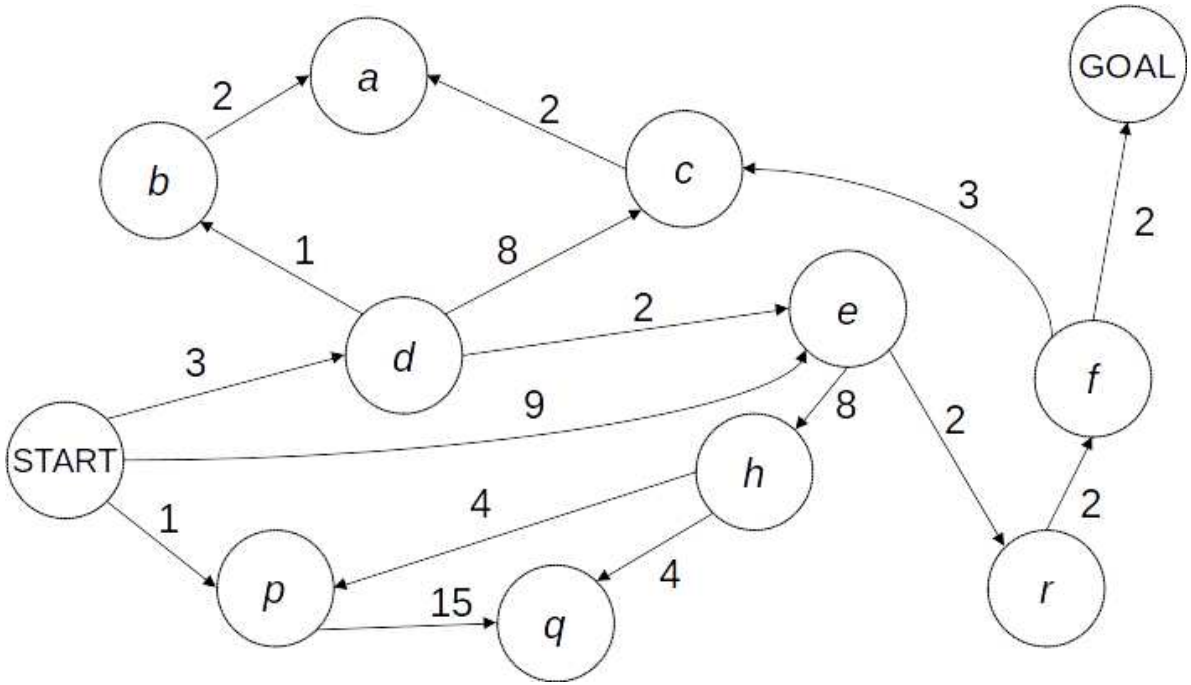
Quiz 2

Care este rezultatul cautarii in latime, adancime si cu cost uniform pentru graful de mai jos? Mentionati solutiile.



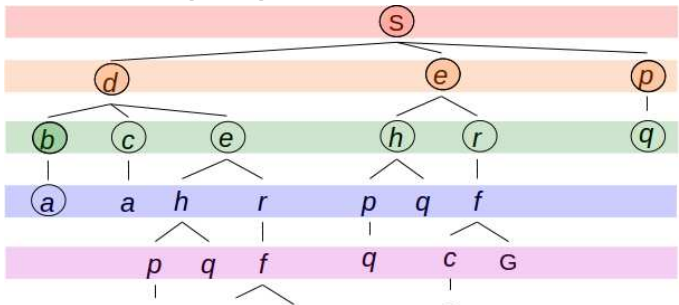
Quiz 2

Care este rezultatul cautarii in latime, adancime si cu cost uniform pentru graful de mai jos? Mentionati solutiile.



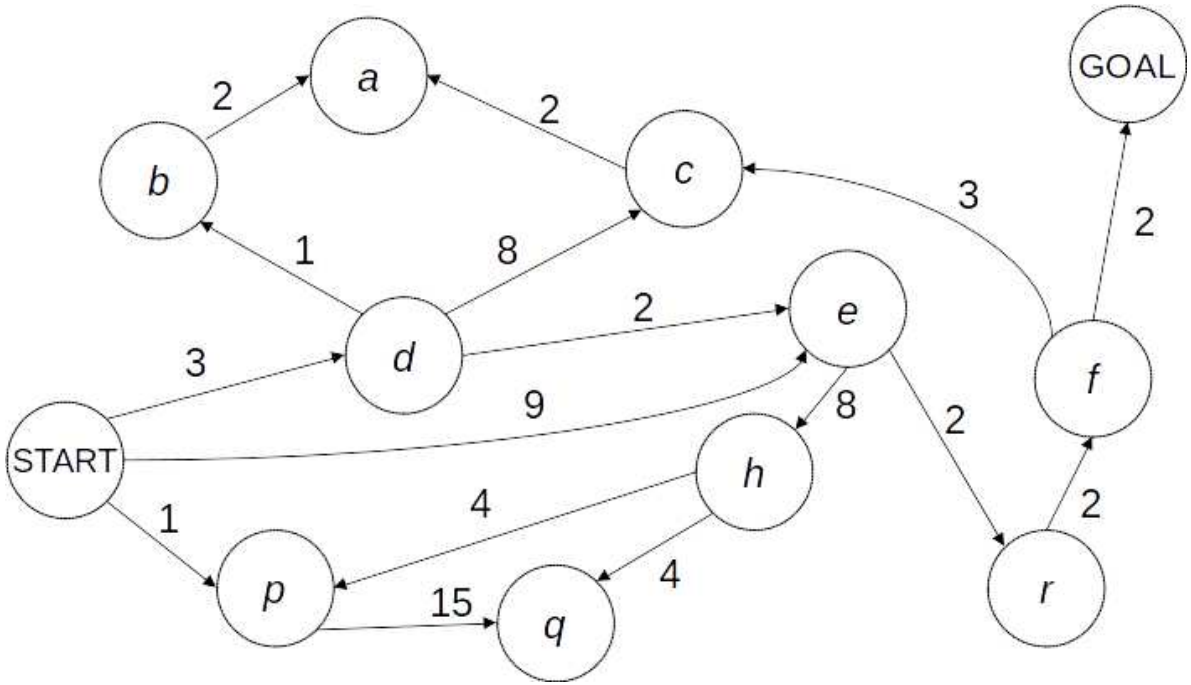
Obs: nodurile incercuite sunt cele expandate. !!! Nu sunt marcate toate nodurile expandate.

BFS: S - e - r - f - G



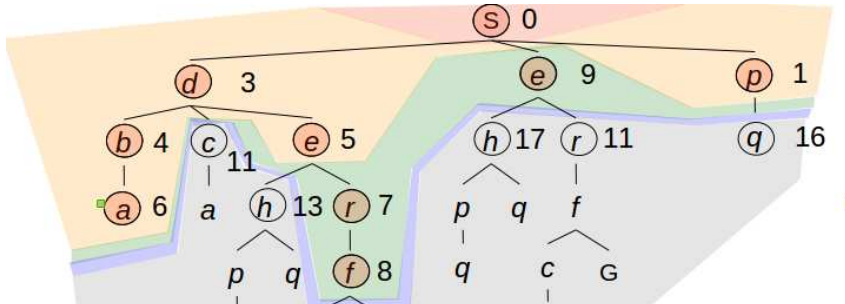
Quiz 2

Care este rezultatul cautarii in latime, adancime si cu cost uniform pentru graful de mai jos? Mentionati solutiile.



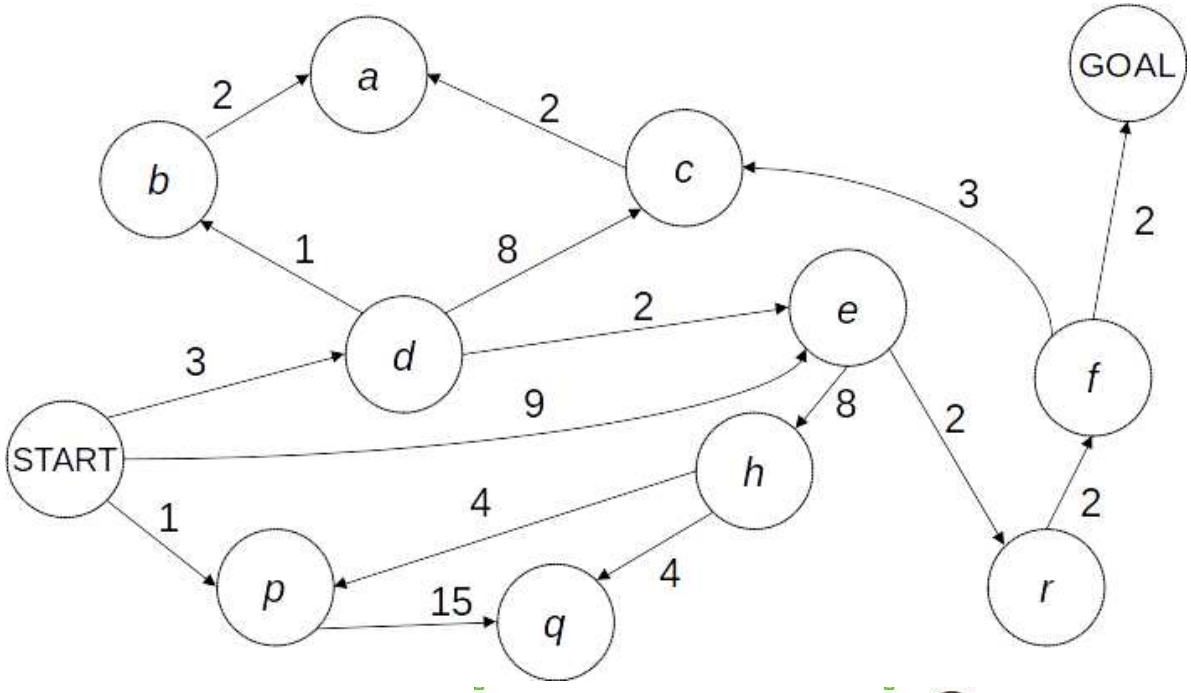
Obs: nodurile incercuite sunt cele expandate. !!! Nu sunt marcate toate nodurile expandate.

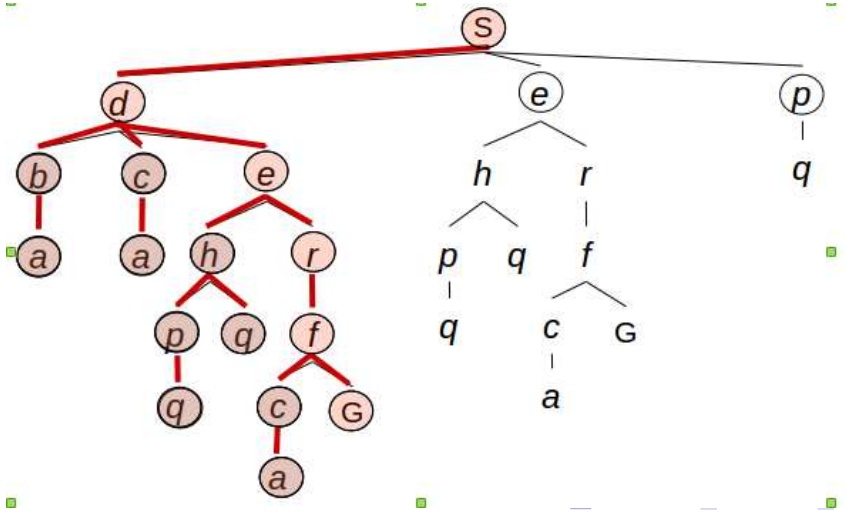
UCS: S - d - e -r - f - G : cost 11



Quiz 2

Care este rezultatul cautarii in latime, adancime si cu cost uniform pentru graful de mai jos? Mentionati solutiile.





DFS: S- d - e - r - f - G

Cautare informata. Informed search

Foloseste cunostinte specifice problemei, pe langa definirea problemei - dezirabilitatea unui nod

Jocul de cautare rece/cald

http://theory.stanford.edu/~amitp/GameProgramming/Variatio

https://www.movingai.com/SAS/SUB/

https://www.movingai.com/SAS/ASM/

http://qiao.github.io/PathFinding.js/visual/

Cautare informata. Informed search

Foloseste cunostinte specifice problemei, pe langa definirea problemei - dezirabilitatea unui nod

Jocul de cautare rece/cald

`http://theory.stanford.edu/~amitp/GameProgramming/Variatio`

`https://www.movingai.com/SAS/SUB/`

`https://www.movingai.com/SAS/ASM/`

`http://qiao.github.io/PathFinding.js/visual/`

- Best-first search - instanta a problemei TREE-SEARCH sau GRAPH-SEARCH in care nodurile sunt expandate pe baza unei **functii de evaluare** f(n).
- functia de evaluare $f(n)$ este o estimare a costului
- **se selecteaza nodul cu evaluarea f cea mai mica**,
- Priority queue - se poate folosi o coada de prioritati pentru a mentine frontiera

Cautare informata. Informed search

Foloseste cunostinte specifice problemei, pe langa definirea problemei - dezirabilitatea unui nod

Jocul de cautare rece/cald

`http://theory.stanford.edu/~amitp/GameProgramming/Variatio`

`https://www.movingai.com/SAS/SUB/`

`https://www.movingai.com/SAS/ASM/`

`http://qiao.github.io/PathFinding.js/visual/`

- Best-first search - instanta a problemei TREE-SEARCH sau GRAPH-SEARCH in care nodurile sunt expandate pe baza unei **functii de evaluare** f(n).
- functia de evaluare $f(n)$ este o estimare a costului
- **se selecteaza nodul cu evaluarea f cea mai mica**,
- Priority queue - se poate folosi o coada de prioritati pentru a mentine frontiera
- **Best-first**?? search - ar insemna ca stim exact ce trebuie facut pentru a ajunge la scop↔ **Seemingly-best-first search**.

Euristici de cautare

$h(n)$ = costul estimat al celei mai scurte cai de la un nod n la scop

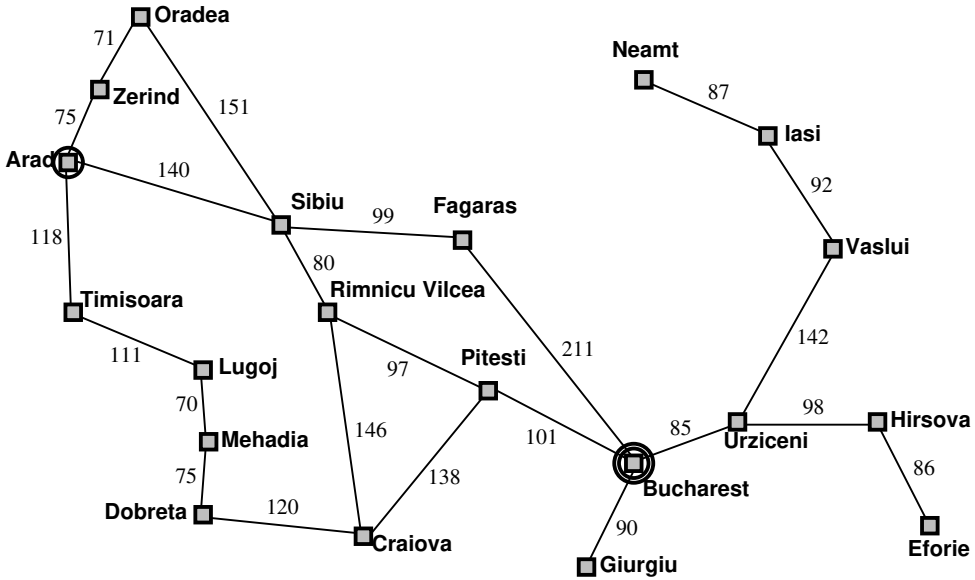
O functie euristica este

- o functie care **estimeaza** cat de aproape de scop e o stare
- construita pentru o anumita problema de cautare

Problema deplasarii in Romania - linia dreapta intre doua orase.

h(n)=0 daca n este chiar scopul

Romania - distantele in linie dreapta pana la Bucuresti

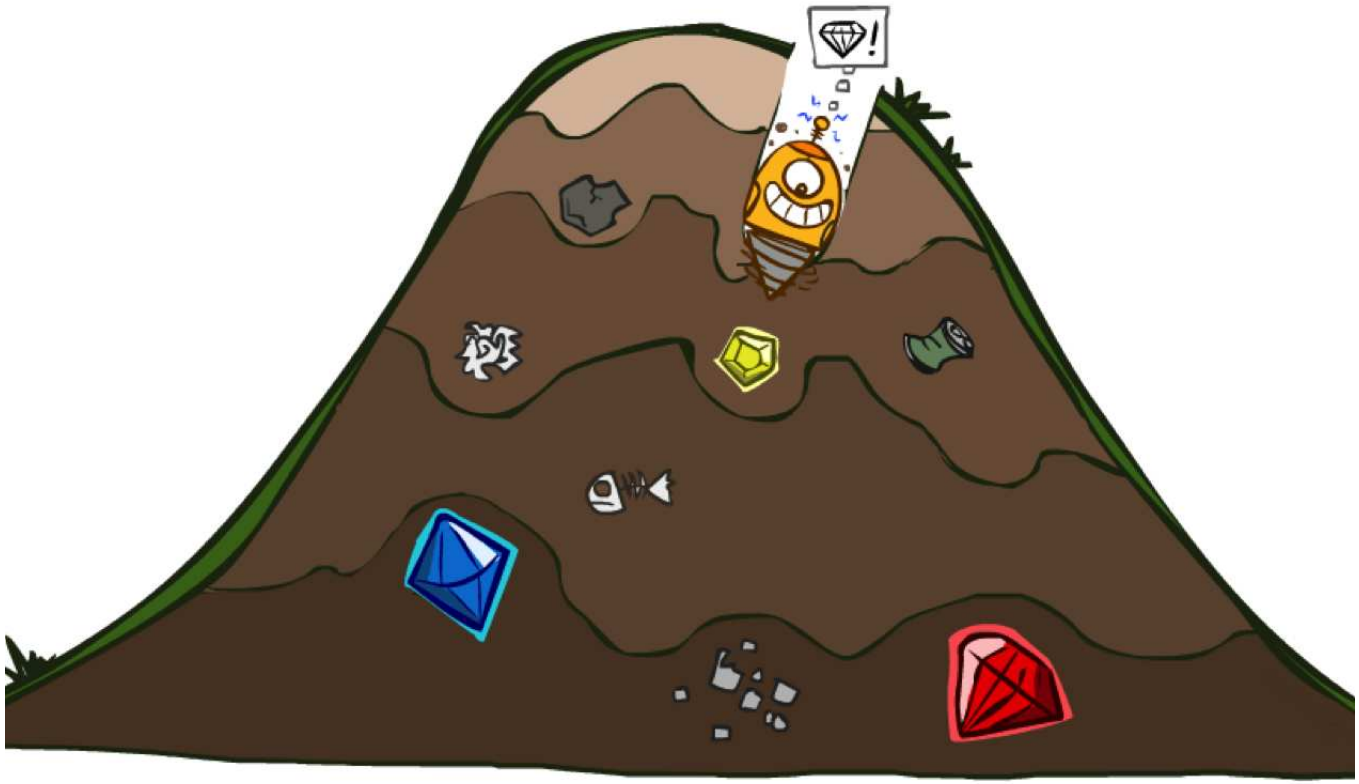


Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-first search

- expandeaza nodul care e cel mai apropiat de scop, considerand ca in acest fel se va ajunge la solutie mai repede
- f(n)=h(n)

Greedy Best-first tree search



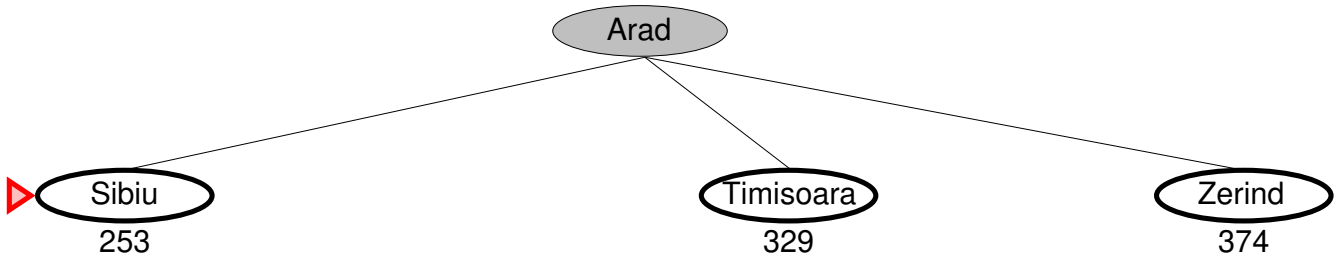
Imagine preluata din curs AI edXedge

Greedy Best-first tree search

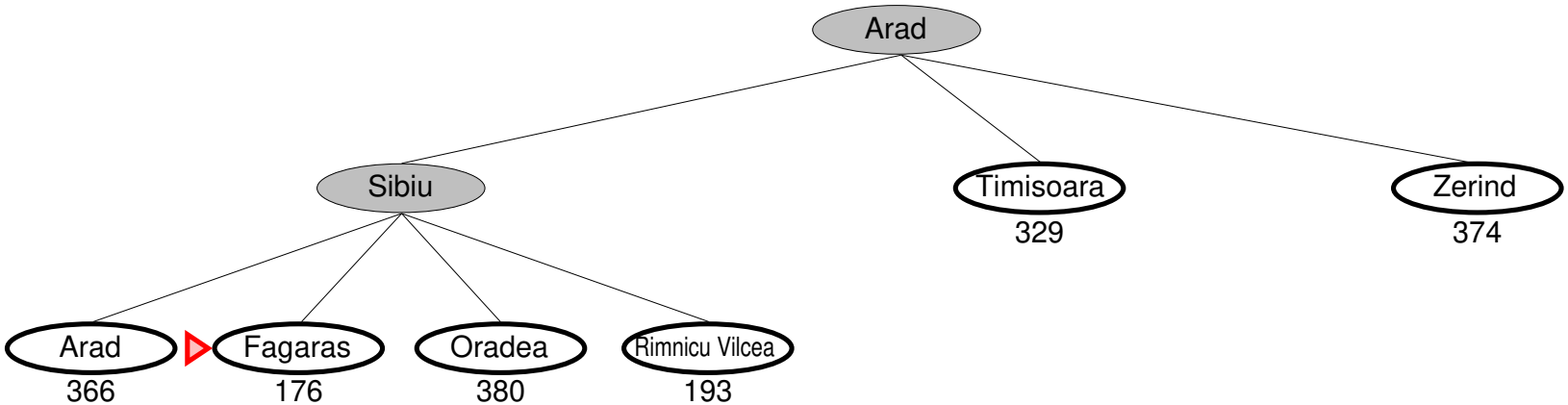


Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Dobreta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

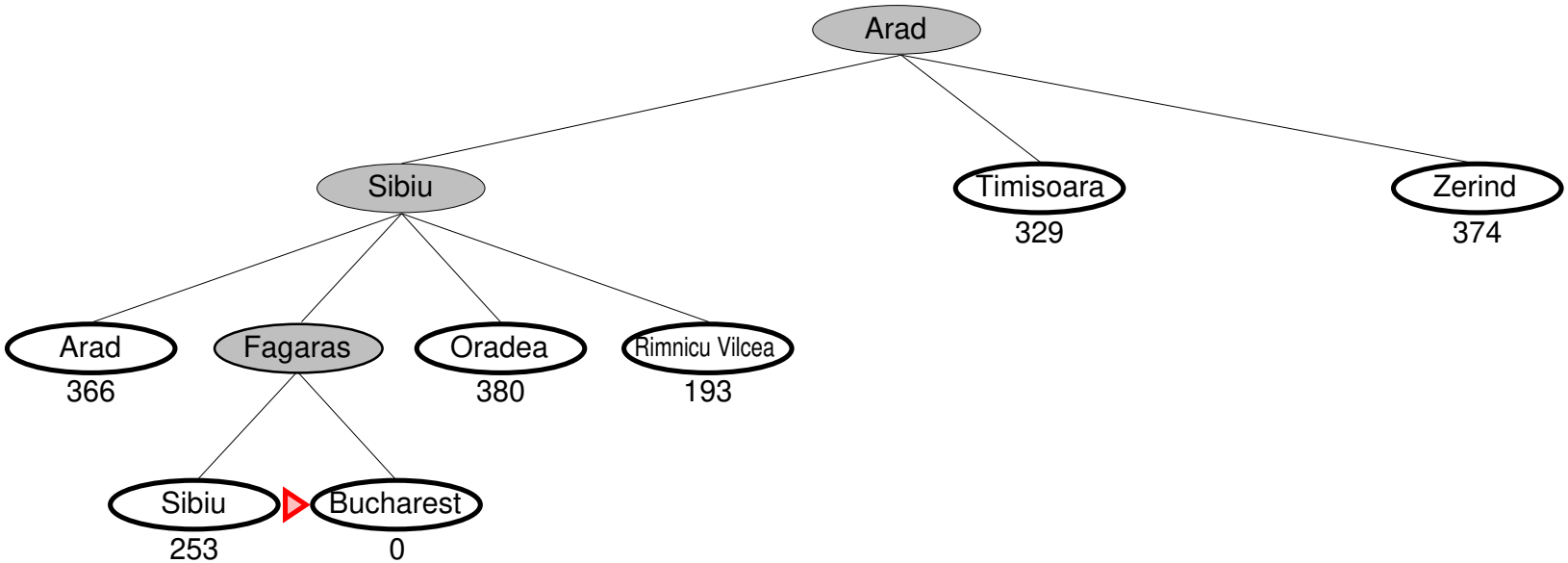
Greedy Best-first tree search



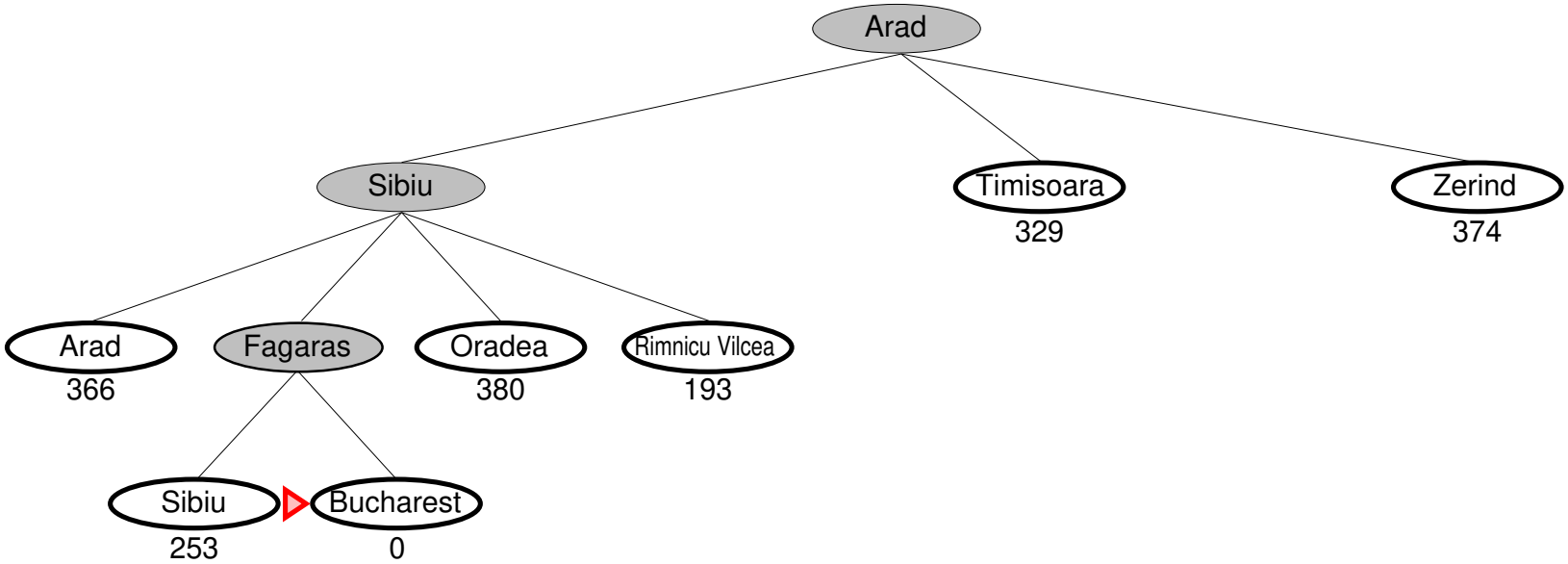
Greedy Best-first tree search



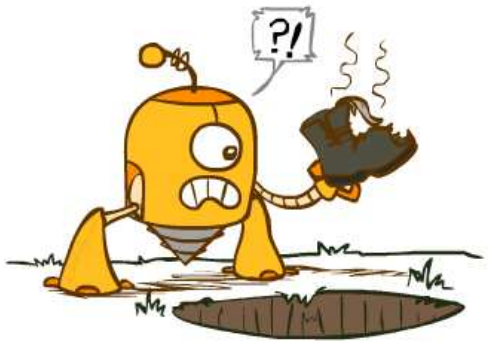
Greedy Best-first tree search



Greedy Best-first tree search



,

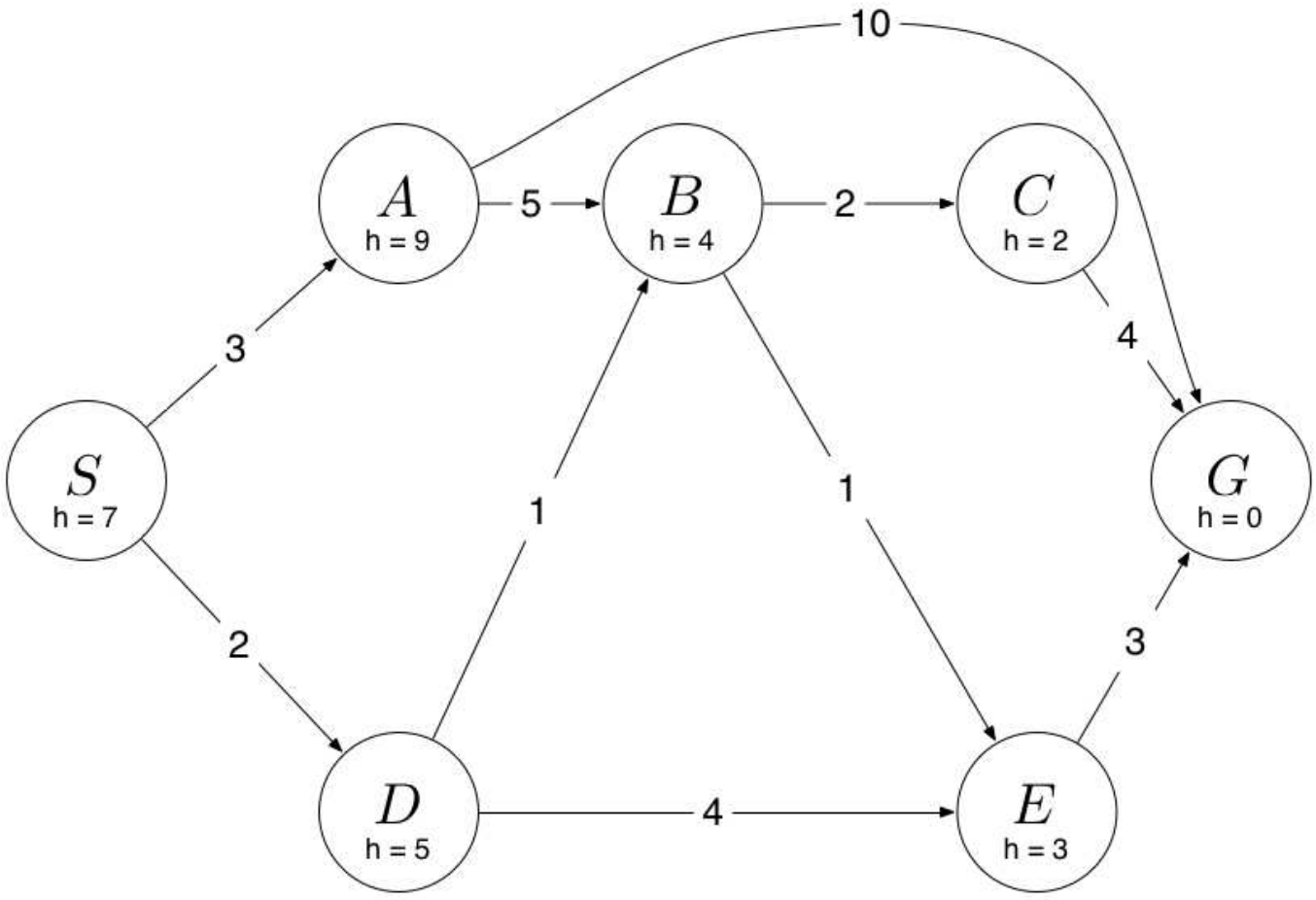


see pacman greedy teaching position

Greedy Best-first search - Observatii

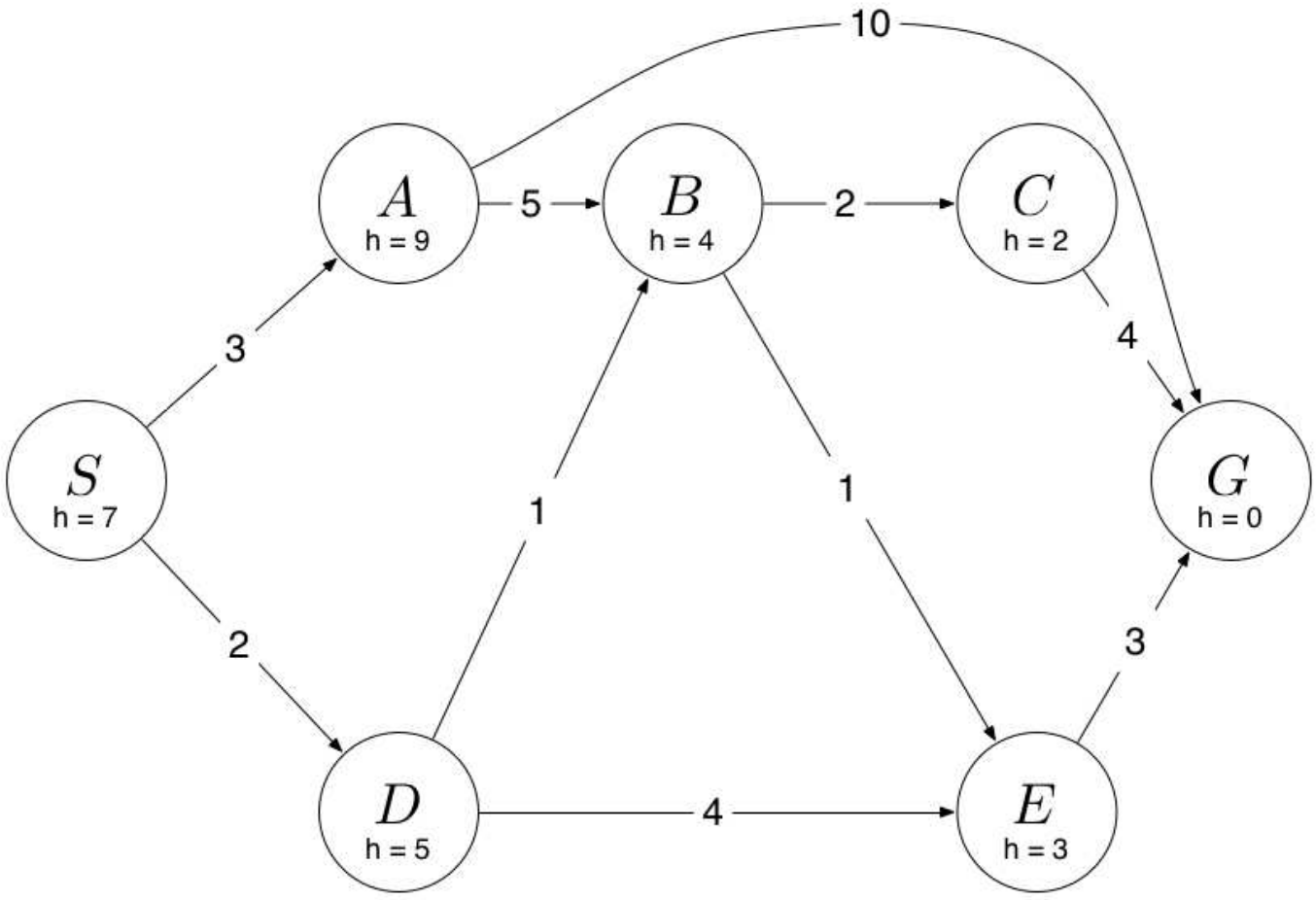
- expandeaza nodul care e cel mai apropiat de scop, considerand ca in acest fel se va ajunge la solutie mai repede
- f(n)=h(n)
- seamana cu **Cautarea in adancime** pentru ca prefera sa urmeze o singura cale catre scop, dar va reveni inapoi cand esueaza
- nu e **optimal**, nu e **complet**, $O(b^m)$ time and space complexity, unde m este adancimea maxima a spatiului de cautare
- cu o euristica buna complexitatea se poate reduce semnificativ

Greedy search



- $S \to A \to G$
- $S \to A \to B \to C \to G$
- $S \to D \to B \to C \to G$
- $S \to D \to B \to E \to G$
- $S \to D \to E \to G$

Greedy search



- $S \rightarrow A \rightarrow G$
- $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$
- $S \rightarrow D \rightarrow B \rightarrow C \rightarrow G$
- $S \rightarrow D \rightarrow B \rightarrow E \rightarrow G$
- $S \rightarrow D \rightarrow E \rightarrow G$

Greedy search - Probleme

Expandeaza prima data ceea ce **pare** a fi mai bun

http://cs-gw.utcluj.ro/~anca/iia/pacman4.ogv

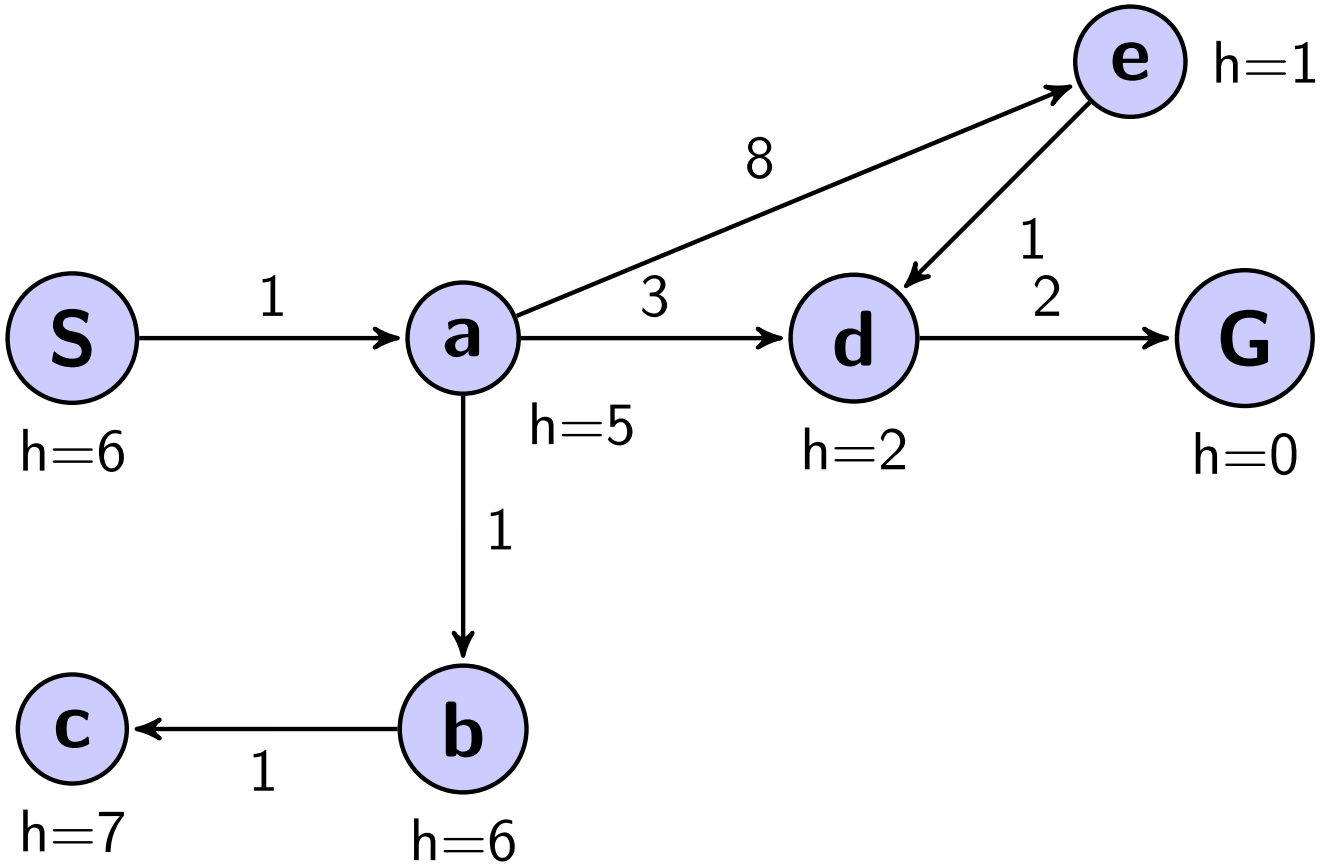
http://www.redblobgames.com/pathfinding/a-star/introductio

Cautarea cu cost uniform + Cautarea Greedy Best-first

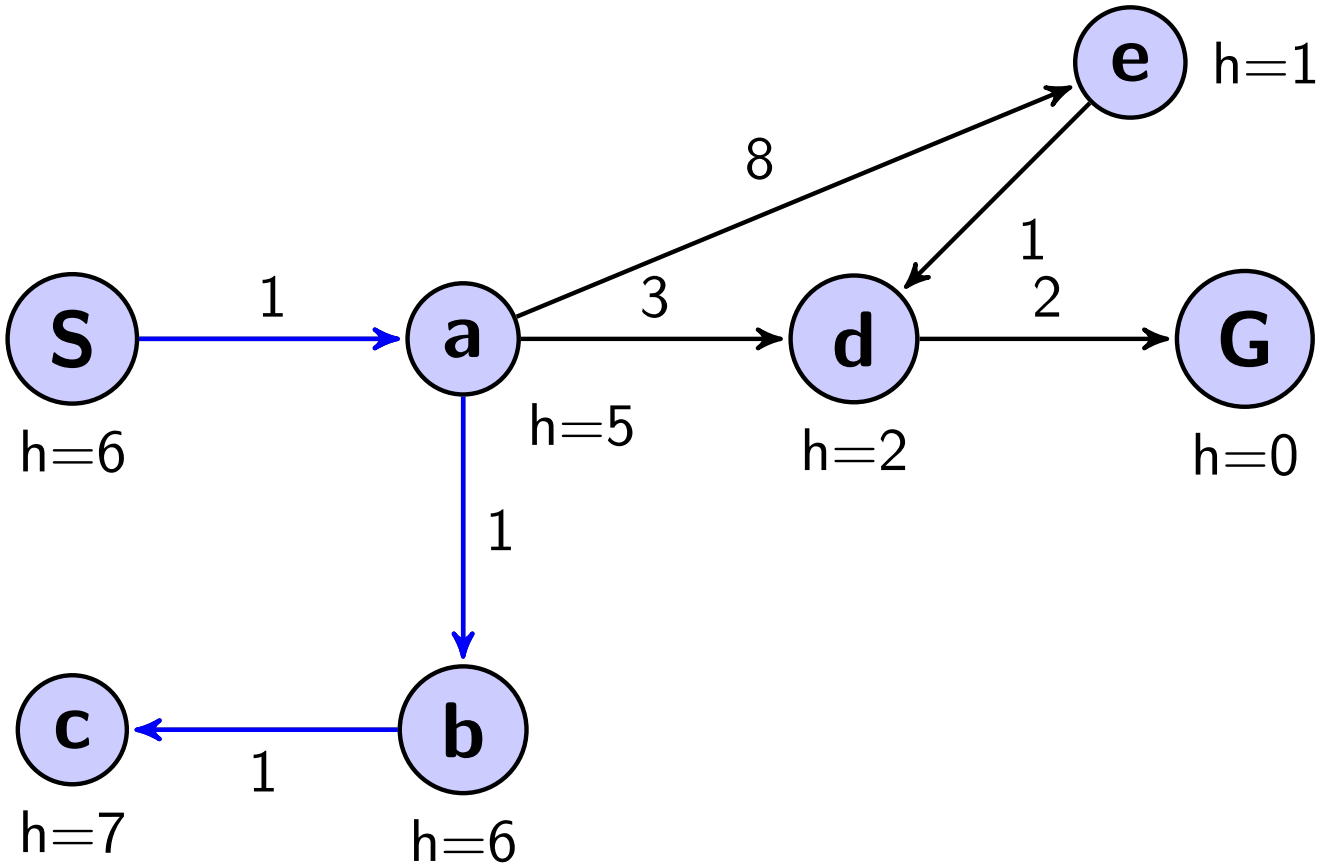


Imagine preluata din curs AI edXedge

Cautarea cu cost uniform + Cautarea Greedy Best-first

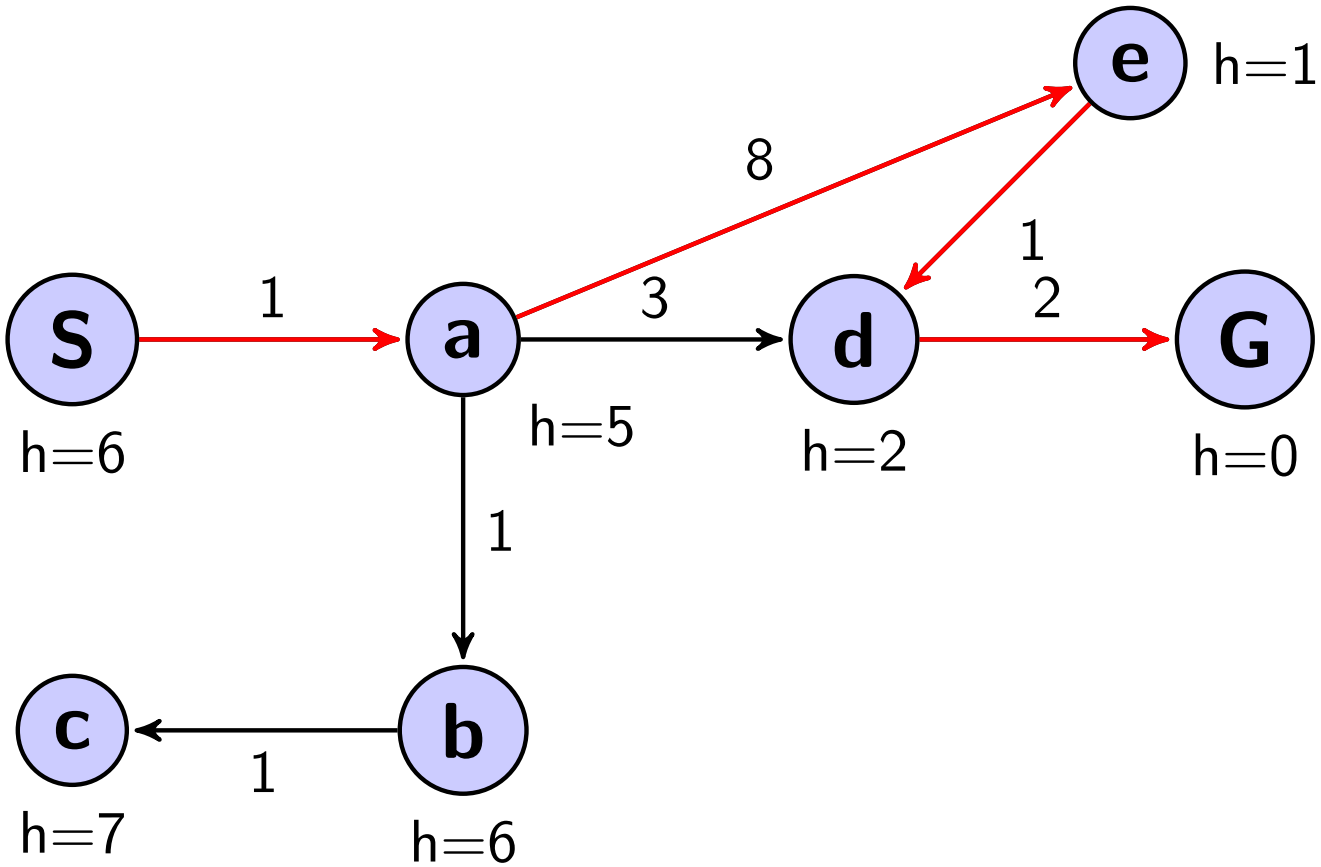


Cautarea cu cost uniform + Cautarea Greedy Best-first



Uniform cost ordoneaza dupa costul caii de pana acum - **backward cost**

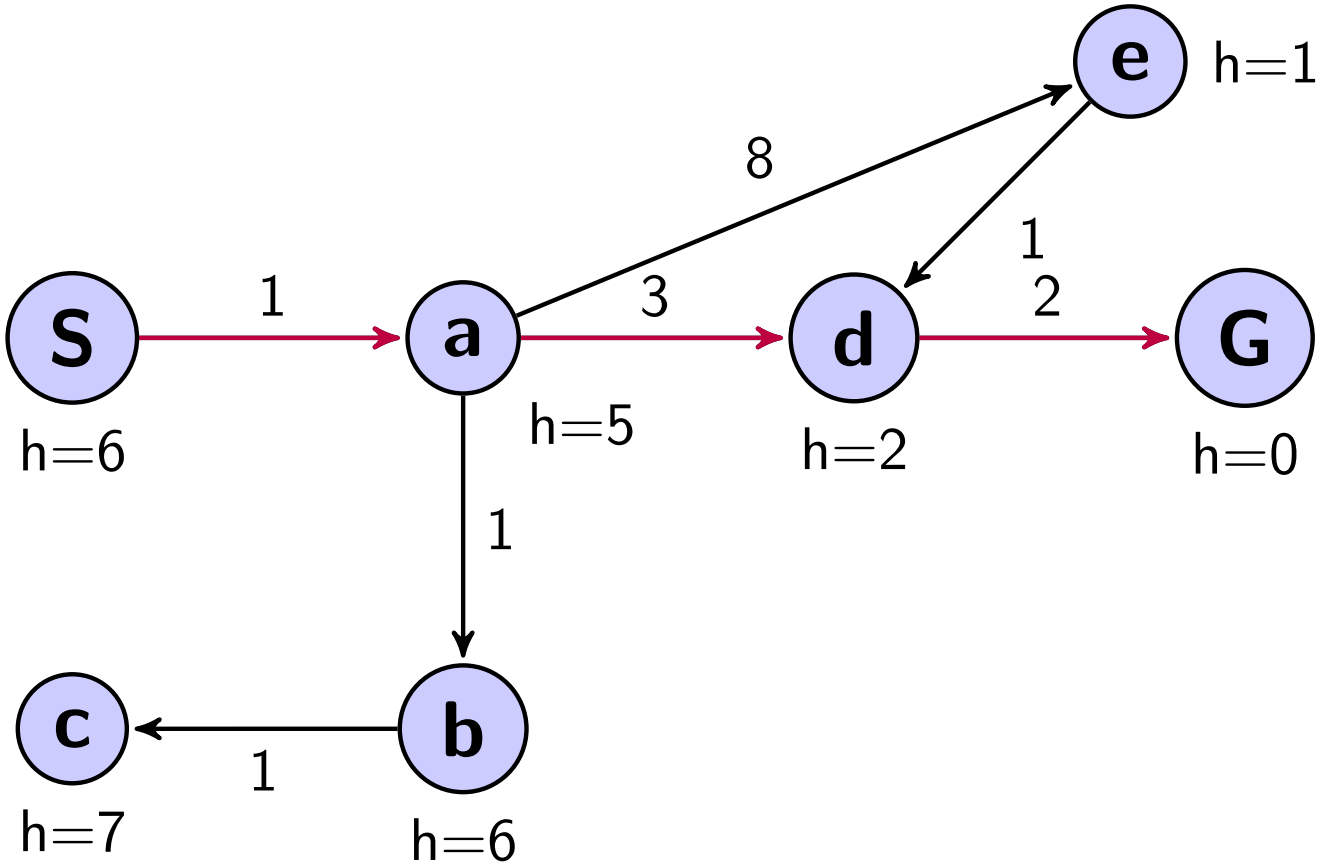
Cautarea cu cost uniform + Cautarea Greedy Best-first



Uniform cost ordoneaza dupa costul caii de pana acum - **backward cost**

Greedy ordoneaza dupa aproprierea de scop - **forward cost**

Cautarea cu cost uniform + Cautarea Greedy Best-first



Uniform cost ordoneaza dupa costul caii de pana acum - **backward cost**

Greedy ordoneaza dupa aproprierea de scop - **forward cost**

A^* evita nodurile care sunt deja scumpe dar si pe cele care par a fi scumpe in viitor

Algoritmul de cautare A^*



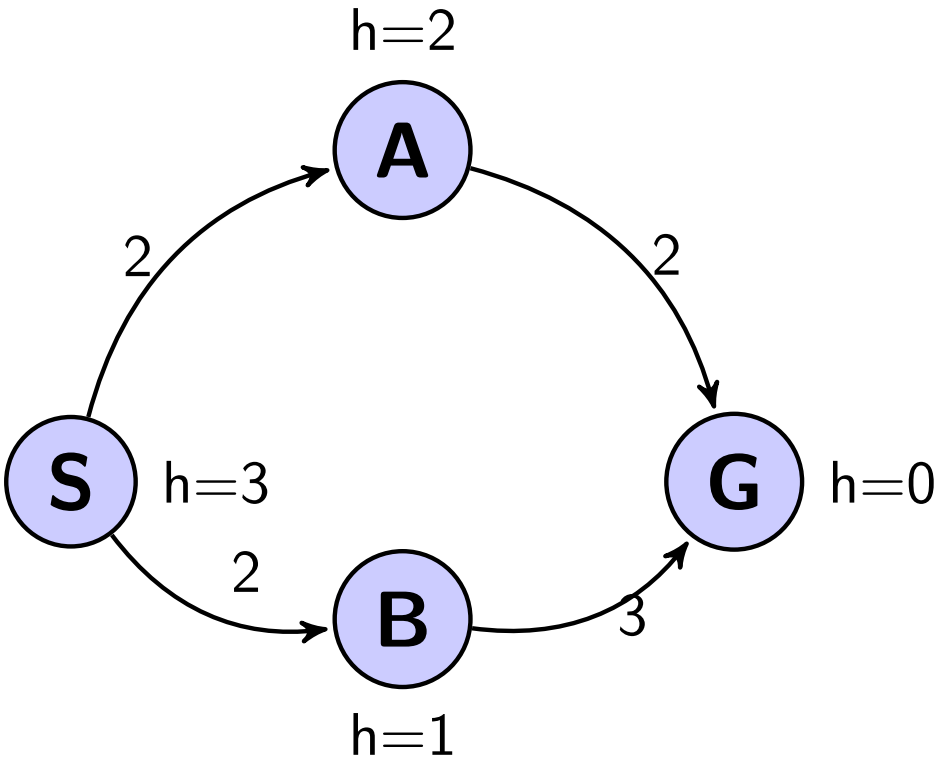
Prima descriere a algoritmului: Peter Hart, Nils Nilsson, Bertram Raphael - Stanford Research Institute - 1968

Imagine preluata din curs AI edXedge

Algoritmul de cautare A^*

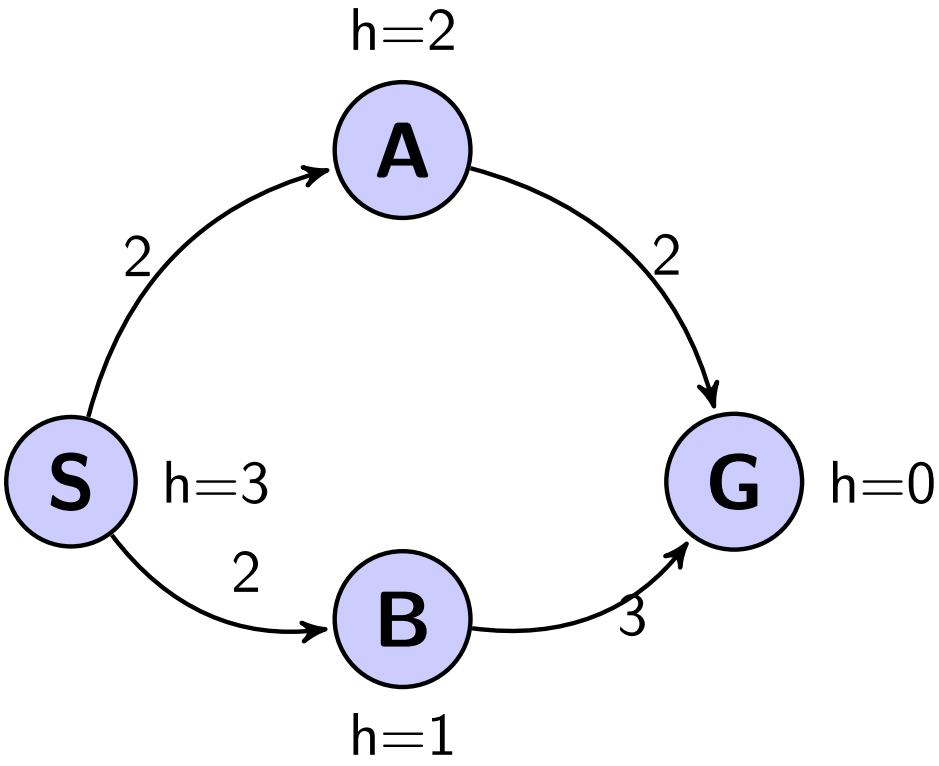
- evitarea expandarii cailor care sunt deja cu cost mare
- functia de evaluare f(n)=g(n)+h(n)
  - g(n)= costul de la nodul initial la nodul n
  - h(n)= costul estimat de la n la goal
  - f(n)=costul total estimat al unei cai catre scop prin nodul n

Cand se termina A^*



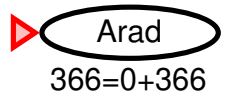
	g	h	f
S	0	3	
S→ A	2	2	4
S→ B	2	1	3
S→ B→ G	5	0	5

Cand se termina A^*

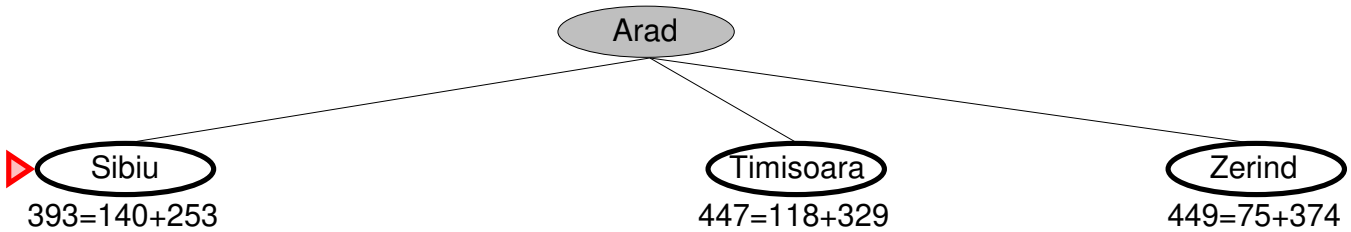


	g	h	f
S	0	3	
S→ A	2	2	4
S→ B	2	1	3
S→ B→ G	5	0	5
S→ A→ G	4	0	4

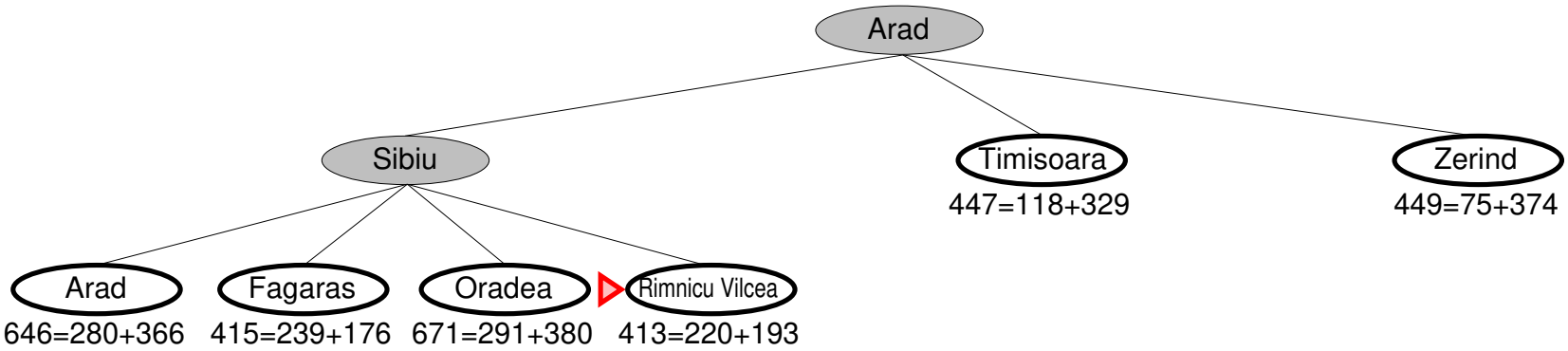
Exemplu A^*



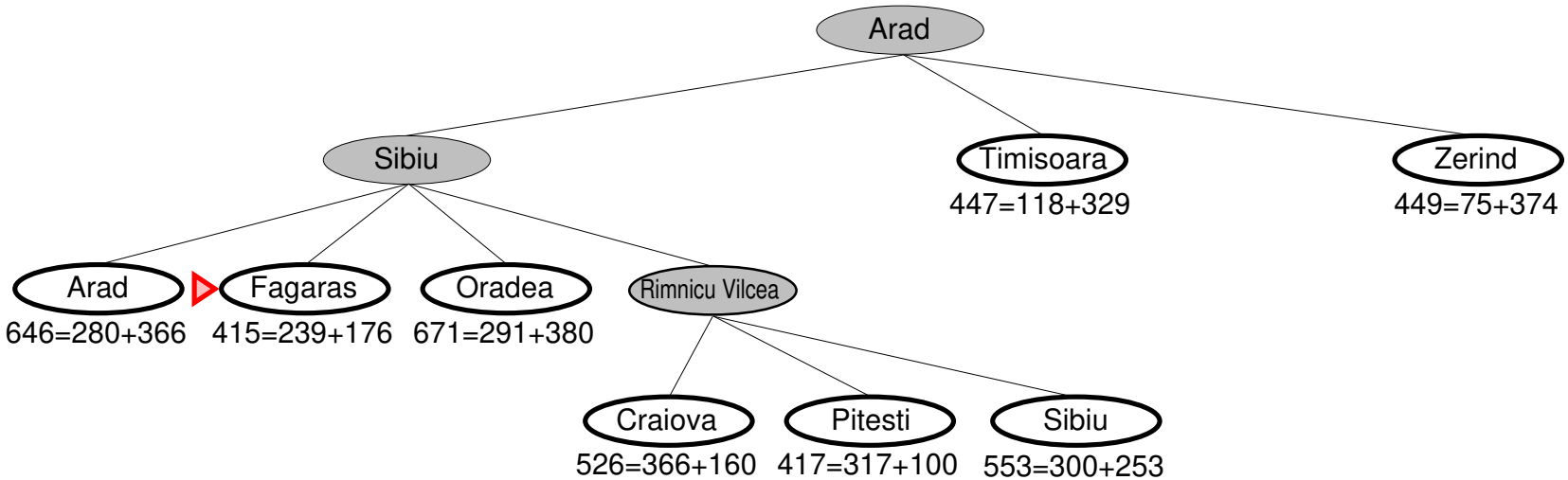
Exemplu A^*


Arad
Sibiu
393=140+253
Timisoara
447=118+329
Zerind
449=75+374

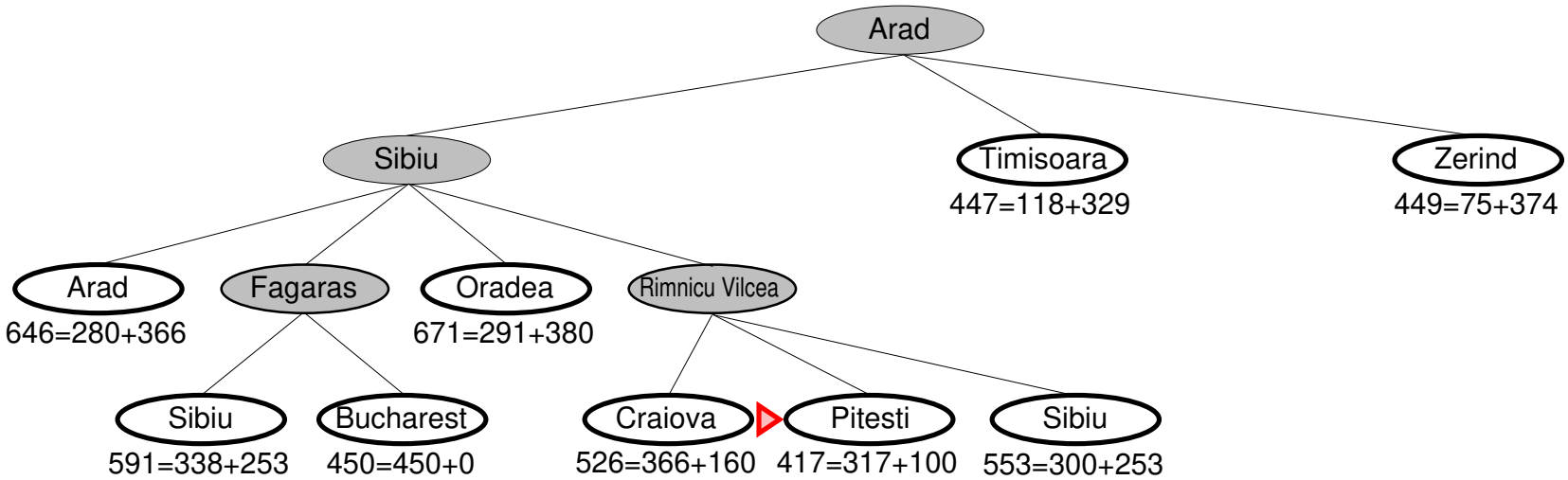
Exemplu A^*


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
413=220+193

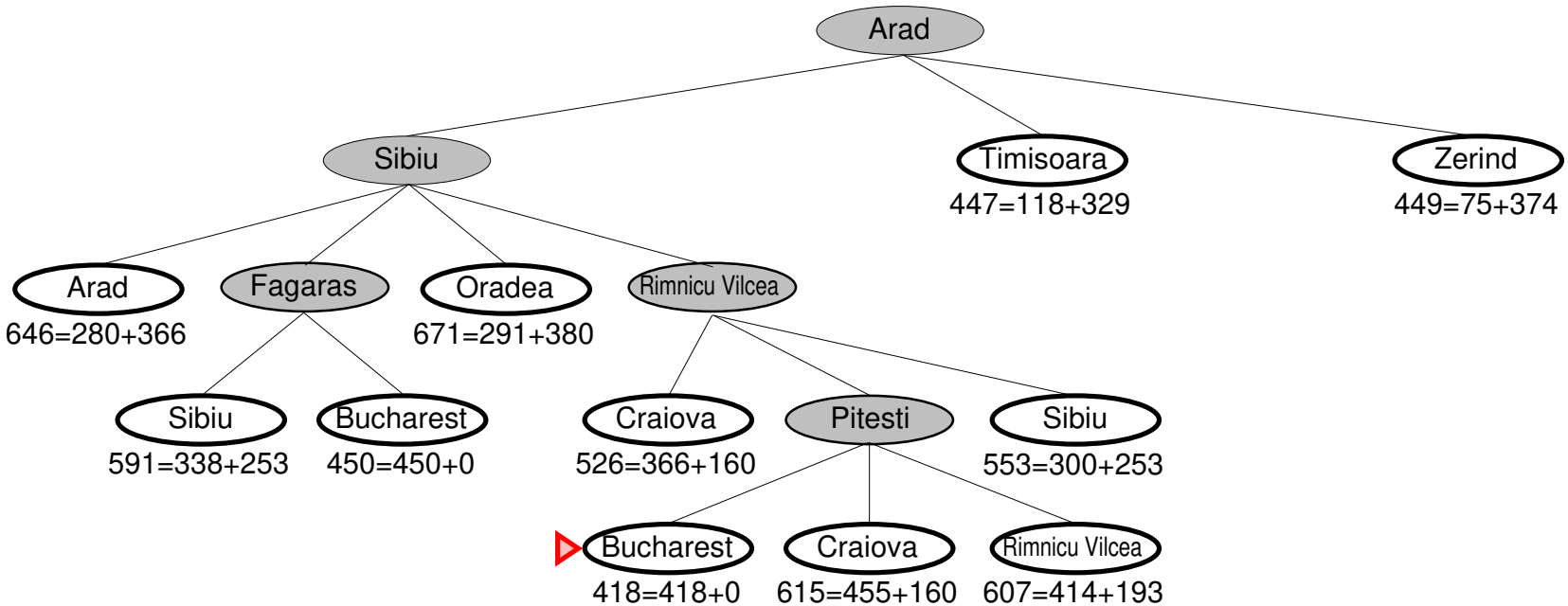
Exemplu A^*


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253

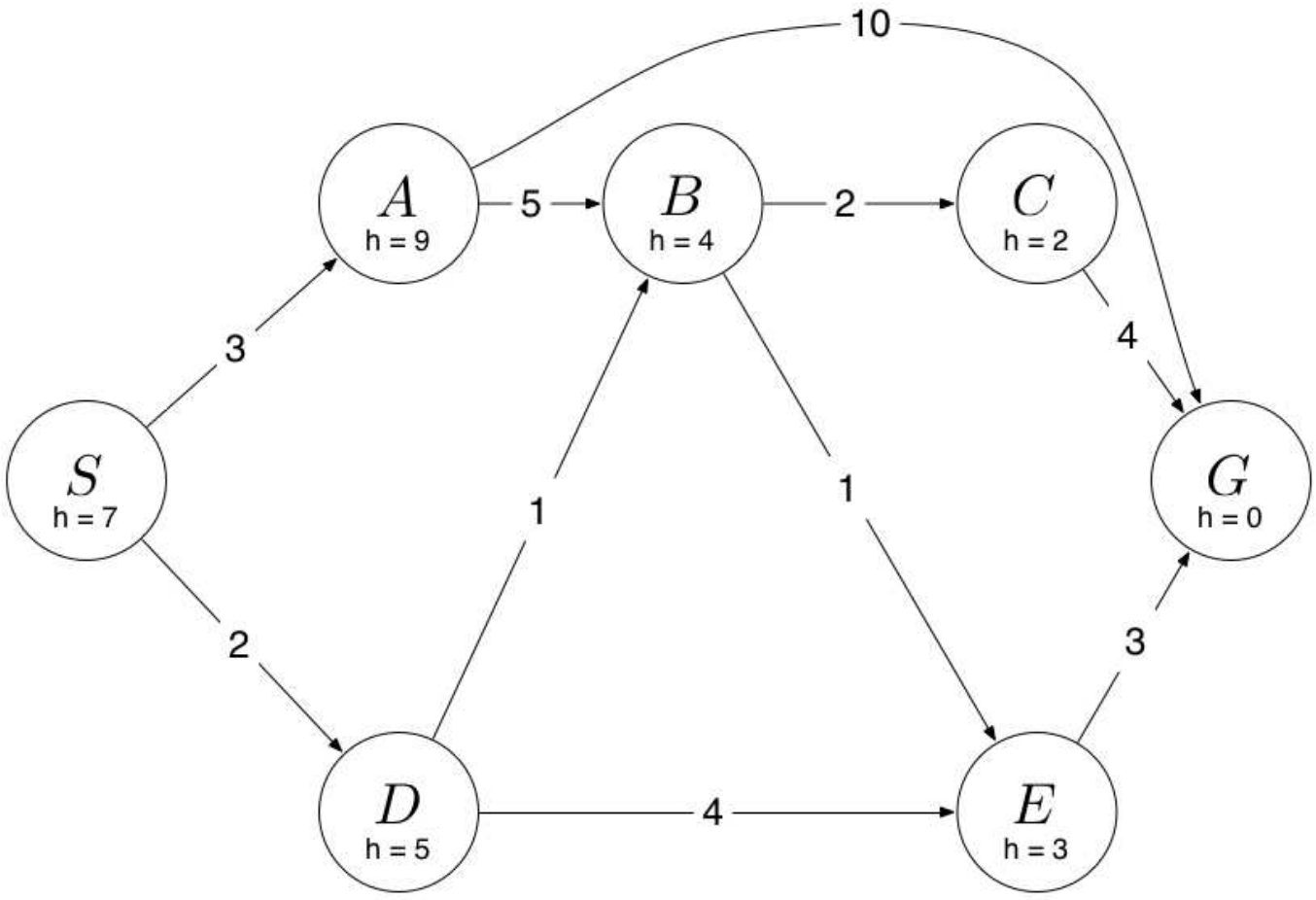
Exemplu A^*


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253

Exemplu A^*

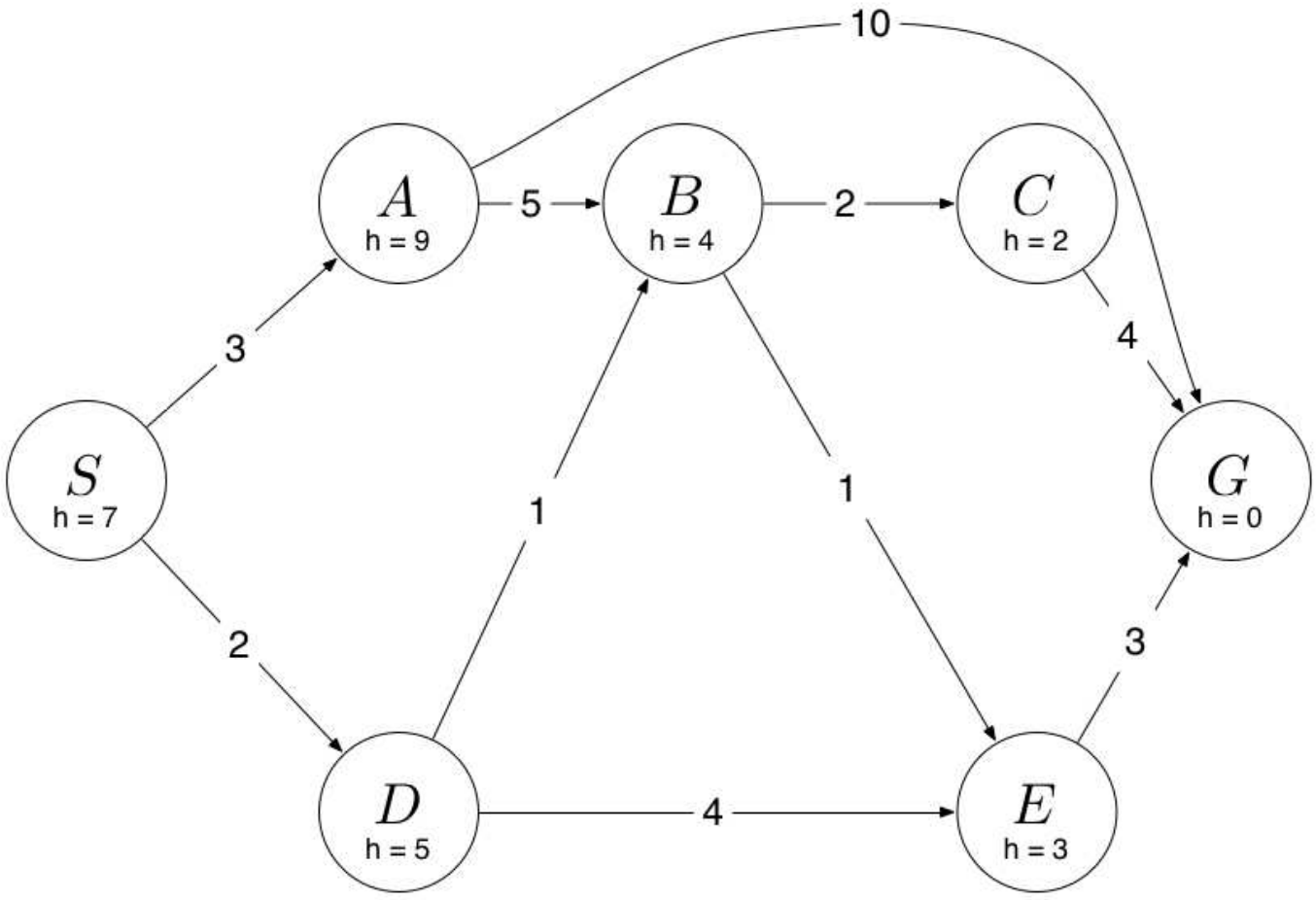

Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
Sibiu
553=300+253
Bucharest
418=418+0
Craiova
615=455+160
Rimnicu Vilcea
607=414+193

A* search



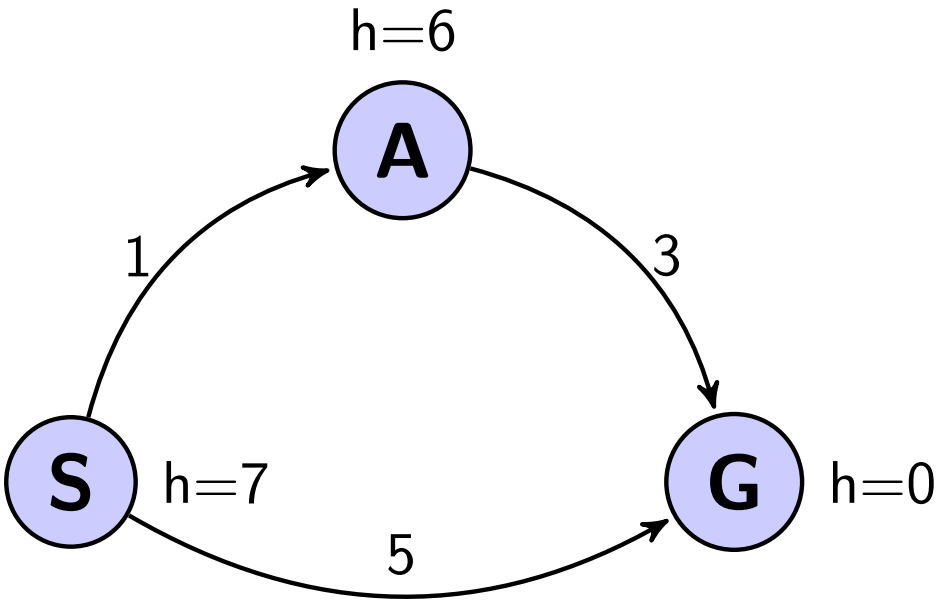
- g((S→ A))=
- h((S→ A))=
- f((S→ A))=
- g((S→ D))=
- h((S→ D))=

A* search



- g((S→ A))= 3
- h((S→ A))= 9
- f((S→ A))= 12
- g((S→ D))= 2
- h((S→ D))= 5

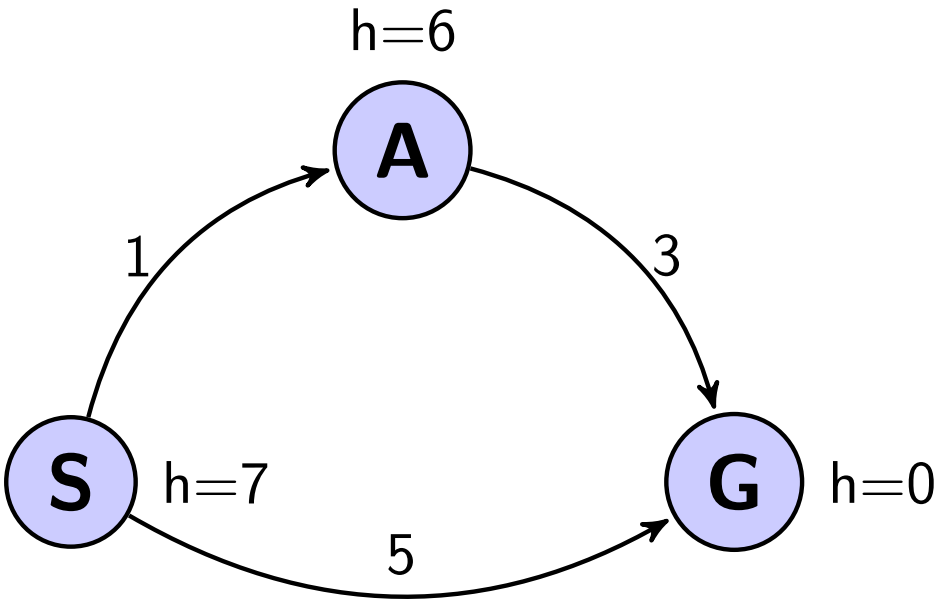
Este A^* optimal?



	g	h	F
S	0	7	7
S$\rightarrow$ A	1	6	7
S$\rightarrow$ G	5	0	5

Solutie: $S \rightarrow G$ neoptimala

Este A^* optimal?



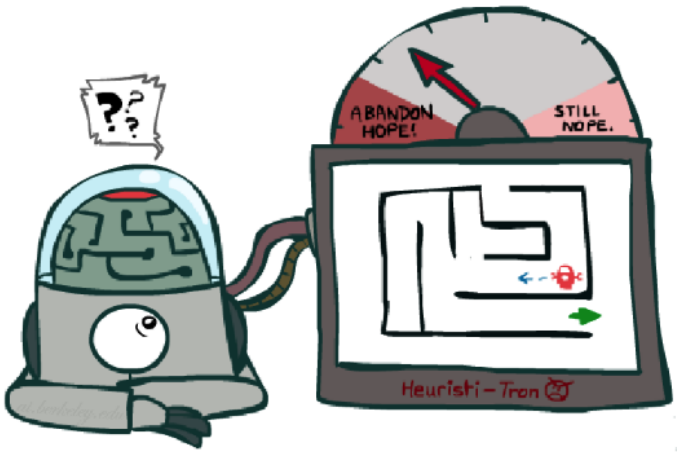
	g	h	F
S	0	7	7
S→ A	1	6	7
S→ G	5	0	5

Solutie: $S \rightarrow G$ neoptimala Euristica pt punctul A supraestimeaza costul din A, facandu-l **fals** neatractiv

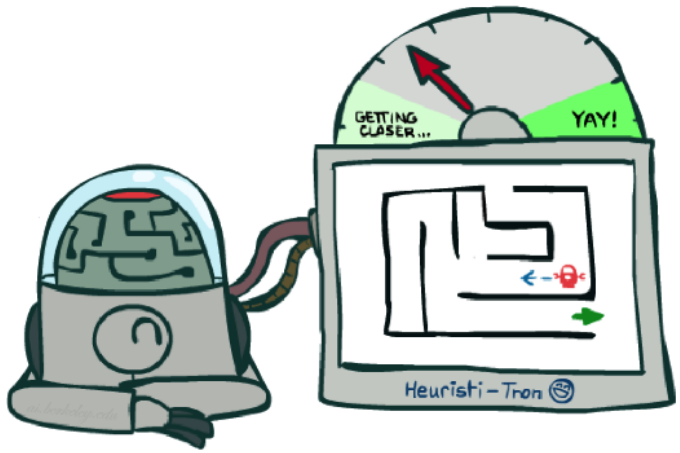
Euristici admisibile

Euristicile admisibile sunt “optimiste“.

Exemplu: Linia dreapta nu supraestimeaza distanta reala a unui drum



euristicile inadmisibile: blocarea planurile bune in frontiera $\rightarrow$ afecteaza optimalitate



euristicile admisibile: inceti-nesc planurile rele dar niciodata nu supraestimeaza costul

Imagine preluata din curs AI edXedge

Euristici admisibile

- o functie euristica este admisibila daca

$$h(n) \leq h^*(n)$$

h^* este costul real pana la cel mai apropiat scop de la n.

- Euristica nu trebuie sa supraestimeze costul real.
- Observatie: $h(Goal) = 0$ si $h(n) \geq 0$

A^* tree-search este optimal daca h(n) este admisibila
A^* graph-search este optimal daca h(n) este consistenta

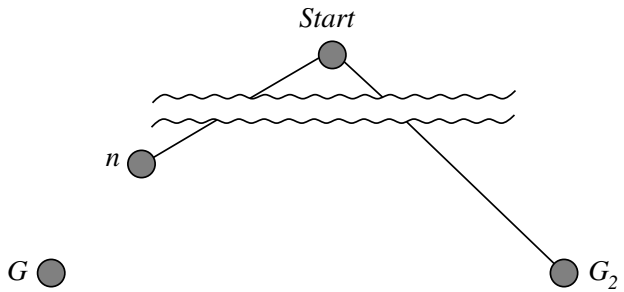
Dem. A^* tree-search este optimal cu euristica admisibila

presupunem:

- un scop suboptimal G_2 a fost generat si pus in frontiera.
- G scop optimal

demonstram: G **este extras din frontiera inaintea lui** G_2

Fie **n** un nod neexpandat de pe o cale catre G.



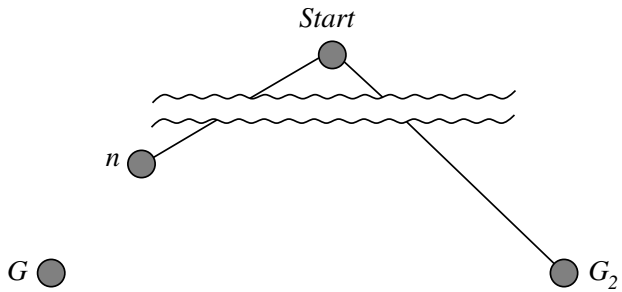
Dem. A^* tree-search este optimal cu euristica admisibila

presupunem:

- un scop suboptimal G_2 a fost generat si pus in frontiera.
- G scop optimal

demonstram: G **este extras din frontiera inaintea lui** G_2

Fie **n** un nod neexpandat de pe o cale catre G.



I. $f(G_2) = g(G_2)$ deoarece $h(G_2) = 0$

$> g(G)$ deoarece G_2 este suboptimal

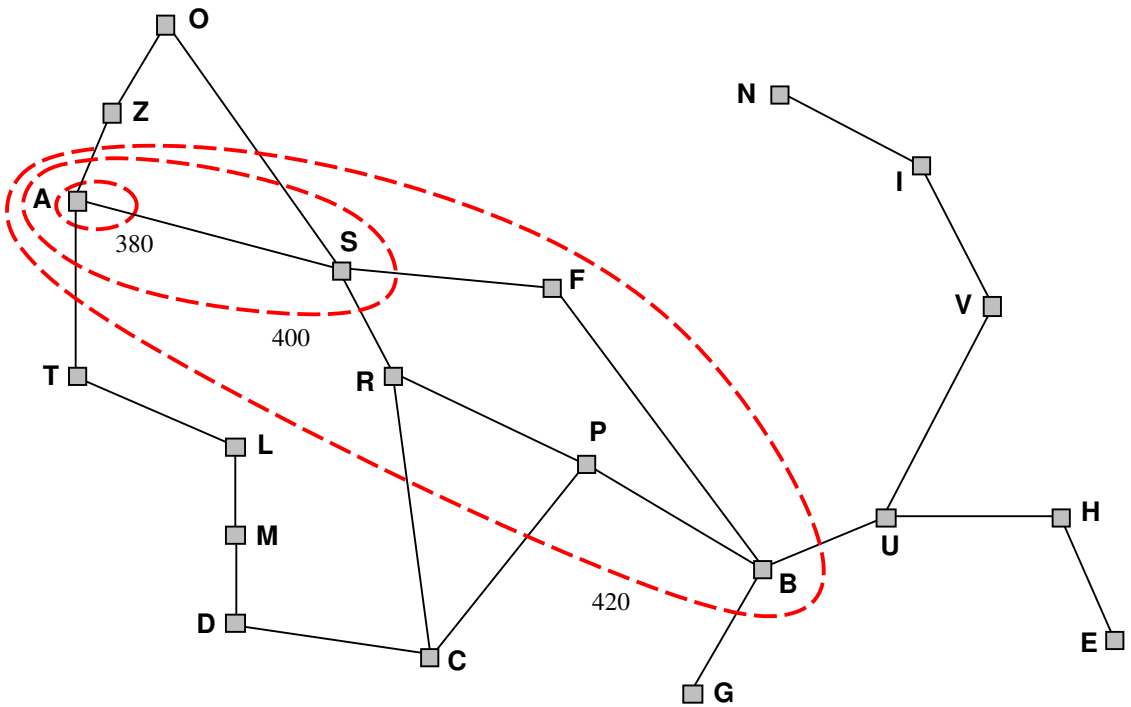
II. $g(G) =$ g(n)+$h^*(n)$, ($h^*(n)$ - costul real de la n la G)

$\geq$ g(n)+h(n) = f(n) deoarece h este admisibila

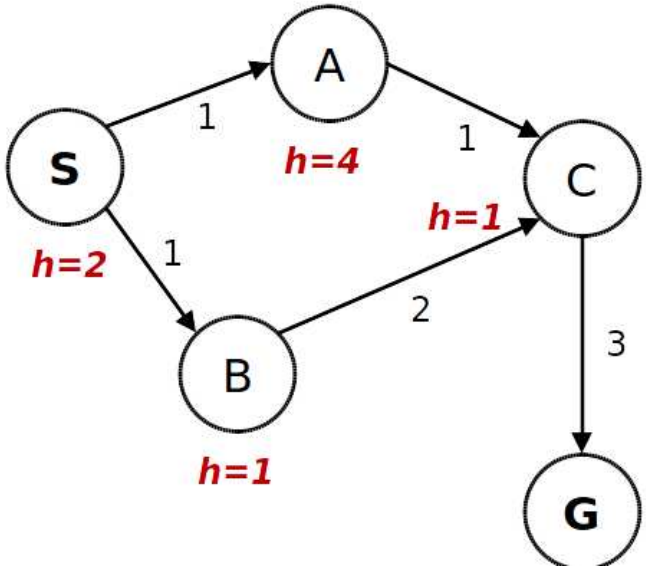
$\Rightarrow f(G_2) > f(n)$, A^* va selecta orice n (stramos al lui G) inaintea lui G_2

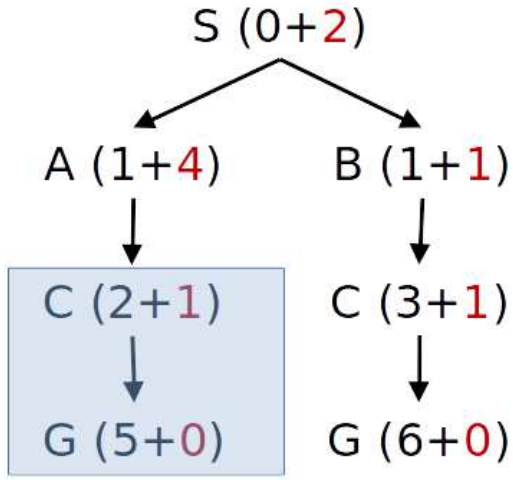
A^* este optimal - continuare

- Lema:A^* expandeaza nodurile in ordinea crescatoare a functiei f
- In mod gradual adauga **contururi f** de noduri (similar cu nivelele adaugate de cautarea in latime)
- Conturul **i** contine toate nodurile cu $f = F_i$, unde $f_i < f_{i+1}$.



Optimalitate A^* graph-search

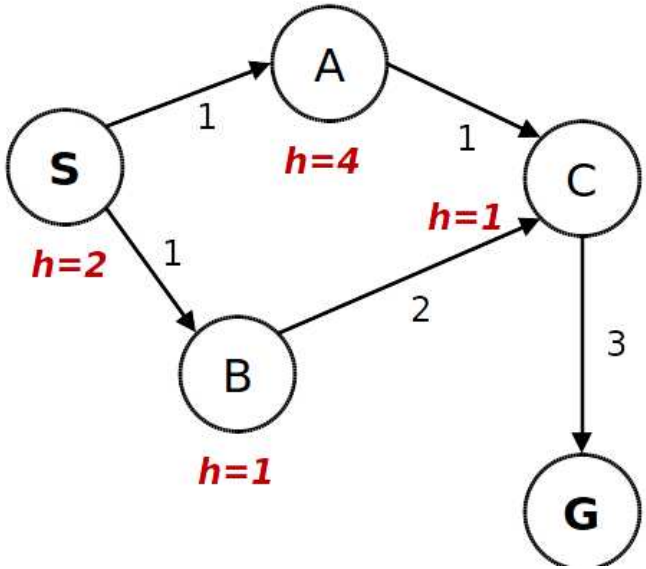


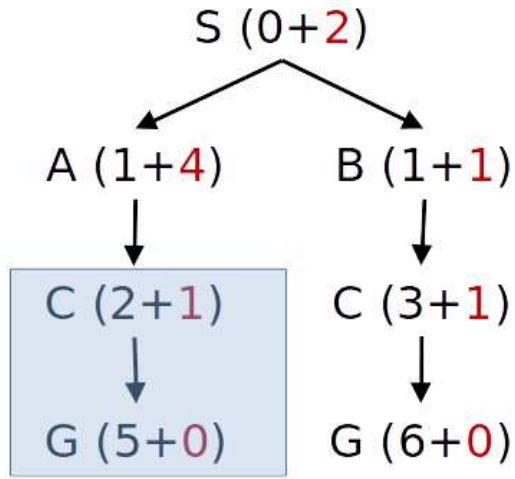


- Solutie gasita: S - B - C - G, cost 6
- Solutie optimala: S- A - C - G, cost 5

De ce?

Optimalitate A^* graph-search





- Solutie gasita: S - B - C - G, cost 6
- Solutie optimala: S- A - C - G, cost 5

De ce? $h(A) - h(C) > c(A \to C)$ Euristica este admisibila, dar nu si consistenta

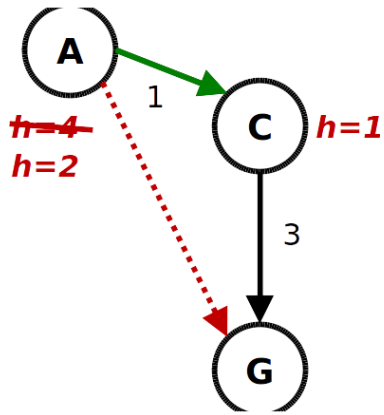
Consistenta (consistency/ monotonicity)

Pentru A^* graph-search sunt necesare euristici consistente.
Euristica consistenta - pentru orice nod n si pentru orice succesor n' a lui n generat prin actiunea a

$$h(n) - h(n') \leq c(n, a, n')$$

Inegalitatea triunghiului

$$h(n) \leq c(n, a, n') + h(n')$$



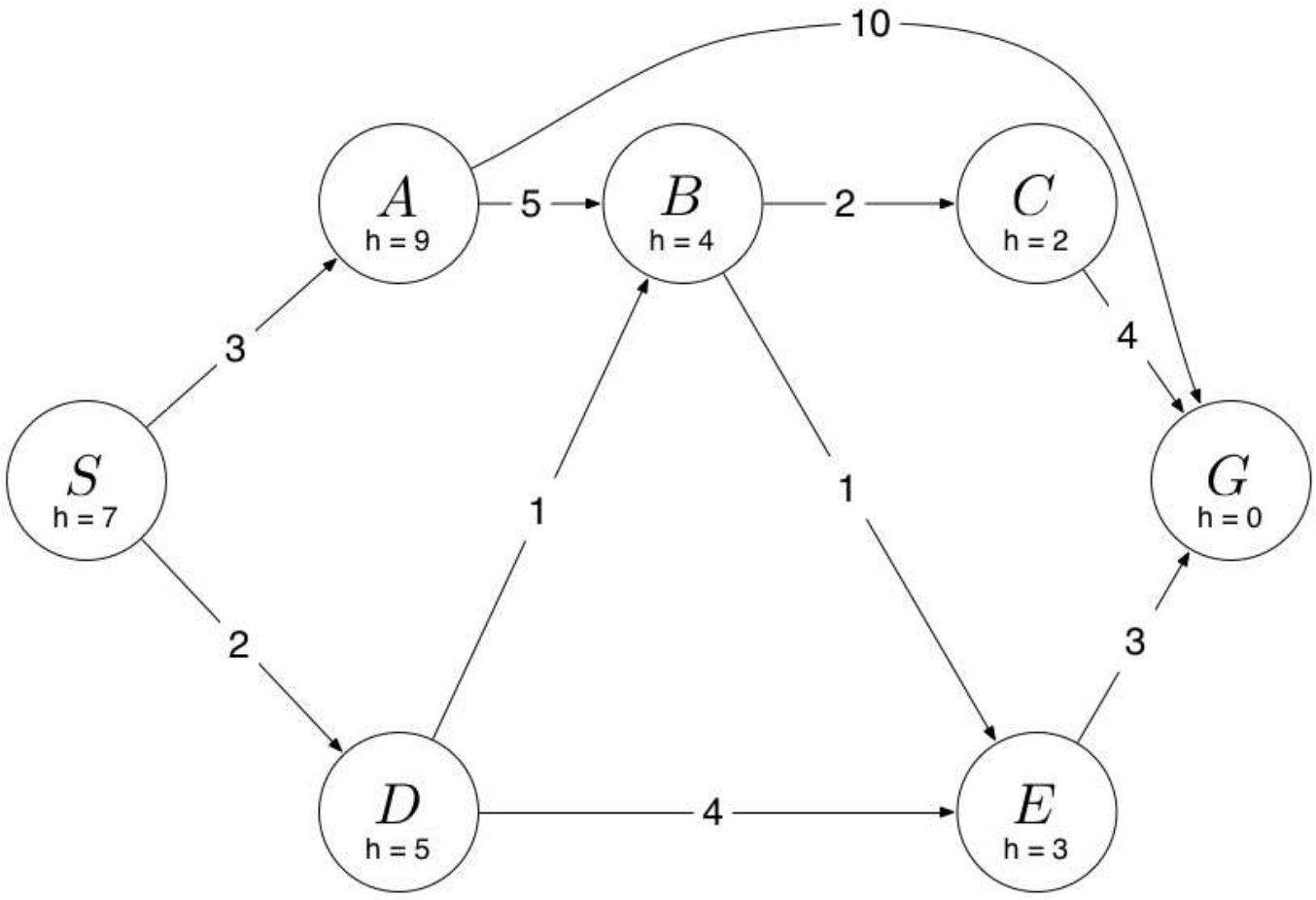
Proprietatile lui A^*

Este complet - daca nu exista o infinitate de noduri cu $f \leq f(G)$
Timp - exponential in **[eroarea relativa a lui h x lungimea solutiei]**
Spatiu - pastreaza toate nodurile in memorie
Optimal - Da: pentru tree-search daca h este admisibila, pentru graph-search daca este consistenta.
Expandeaza:

- toate nodurile cu $f(n) < C^*$, C^* costul solutiei optime
- unele noduri cu $f(n) = C^*$
- nici un nod cu $f(n) > C^*$

A^* - optimally efficient pt orice euristica consistenta = niciun alt algoritm optimal nu expandeaza mai putine noduri decat A^*.

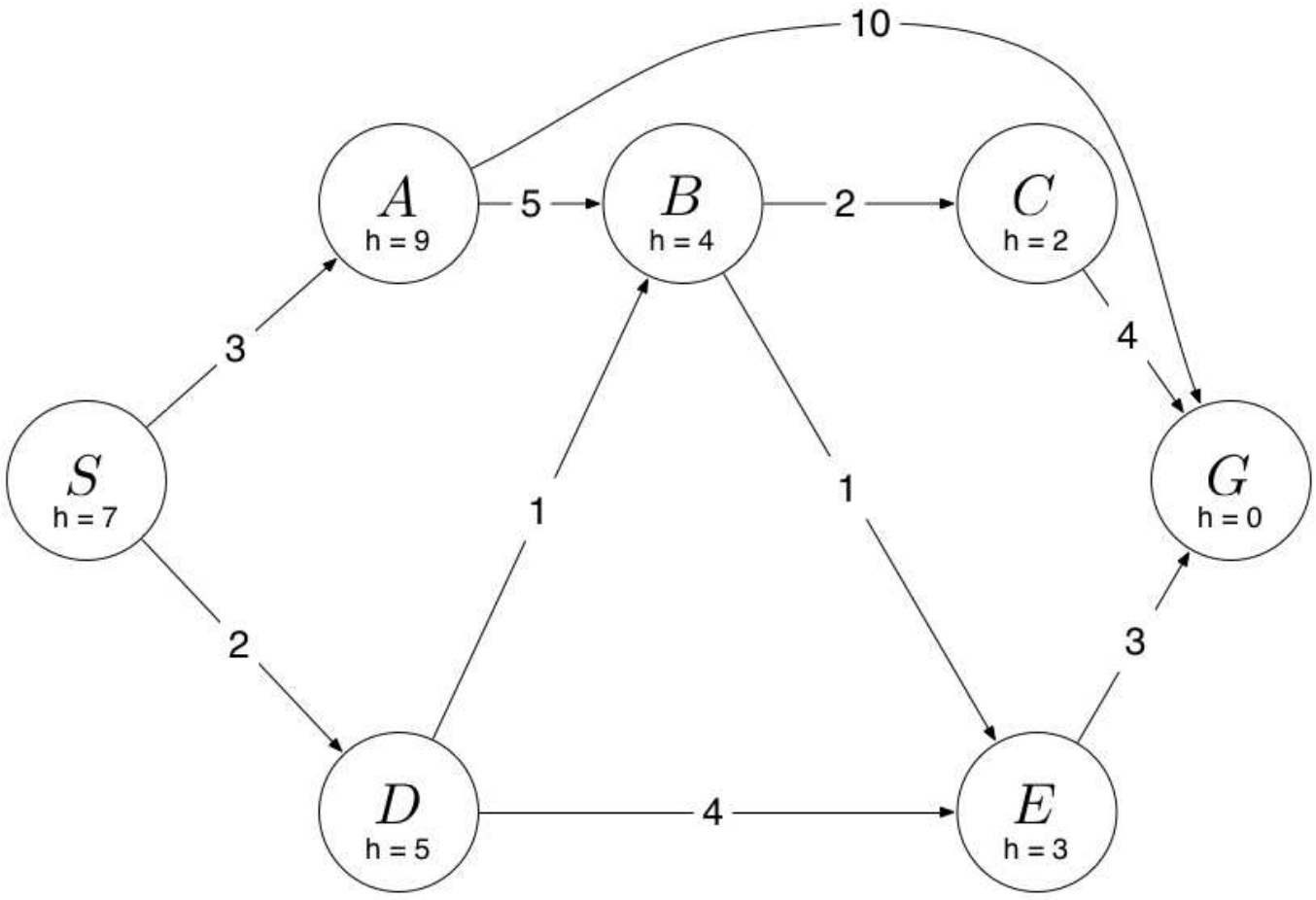
A* search



Euristica este admisibila? Care este solutia identificata?

- $S \to A \to G$
- $S \to A \to B \to C \to G$
- $S \to D \to B \to C \to G$
- $S \to D \to B \to E \to G$

A* search



Euristica este admisibila? Care este solutia identificata?

- $S \to A \to G$
- $S \to A \to B \to C \to G$
- $S \to D \to B \to C \to G$
- $S \to D \to B \to E \to G$

compare Q7 astart-tinySearch + Q7 with bfs + Q7 with dfs

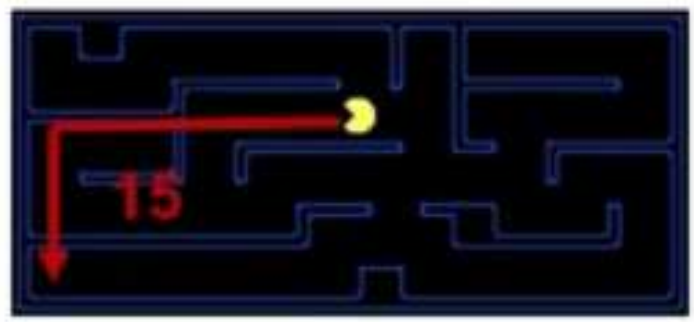
reducerea cerintelor de memorie a A^* $\Rightarrow$:

- iterative-deepening A^*
- recursive best-first search (RBFS)
- memory-bounded A^* and simplified MA^*

Identificarea euristicilor admisibile - Euristici I

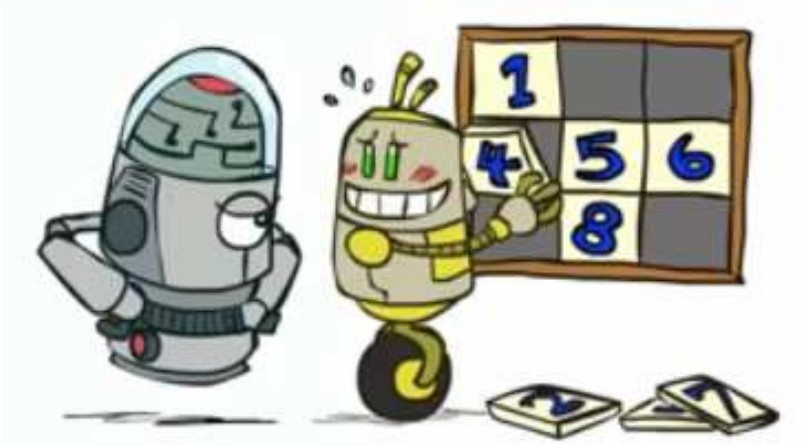
Problema relaxata permite identificarea unor functii euristice admisibile





Euristici pentru 8-puzzle

https://murhafsousli.github.io/8puzzle/



7	2	4
5		6
8	3	1

Start State

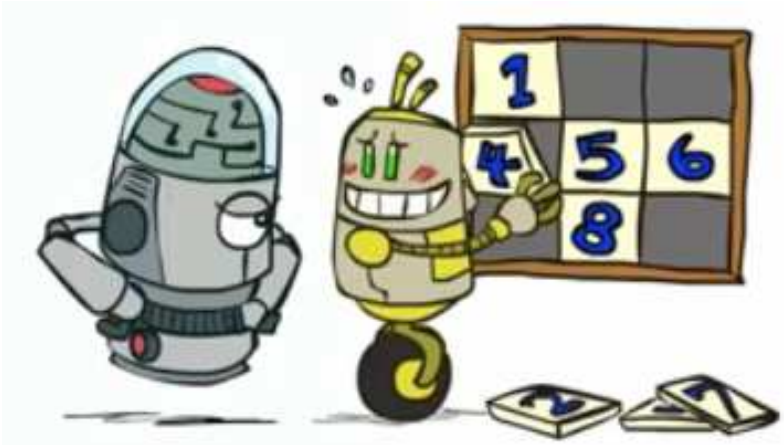
1	2	3
4	5	6
7	8	

Goal State

1. Numarul de piese care nu sunt la locul lor: $h_1(\text{start})=6$

Euristici pentru 8-puzzle

https://murhafsousli.github.io/8puzzle/



7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

1. Numarul de piese care nu sunt la locul lor: $h_1(\text{start})=6$
2. Distanta Manhattan: $h_2(\text{start})=4+0+3+3+1+0+2+1=14$

Daca $h_2(n) \geq h_1(n)$ pentru orice n si ambele sunt admisibile atunci h_2 domina h_1 si este mai buna pentru cautare

Euristici 8-puzzle

Numarul de noduri expandate:

	4 pasi	8 pasi	12 pasi
UCS	112	6300	$3.6x10^6$
Piese in alt loc	13	39	227
Manhatan	12	25	73

Euristici II

Fie problema Pacman cu scopul de a manca toate punctele din labirint. Fiecare actiune are un cost 1. Alegeti euristica/euristicile care sunt admisibile

- numarul total de puncte ramase
- distanta pana la cel mai apropiat punct
- distanta pana la cel mai indepartat punct
- distanta pana la cel mai apropiat punct plus distanta pana la cel mai indepartat punct
- nici una

Euristici II

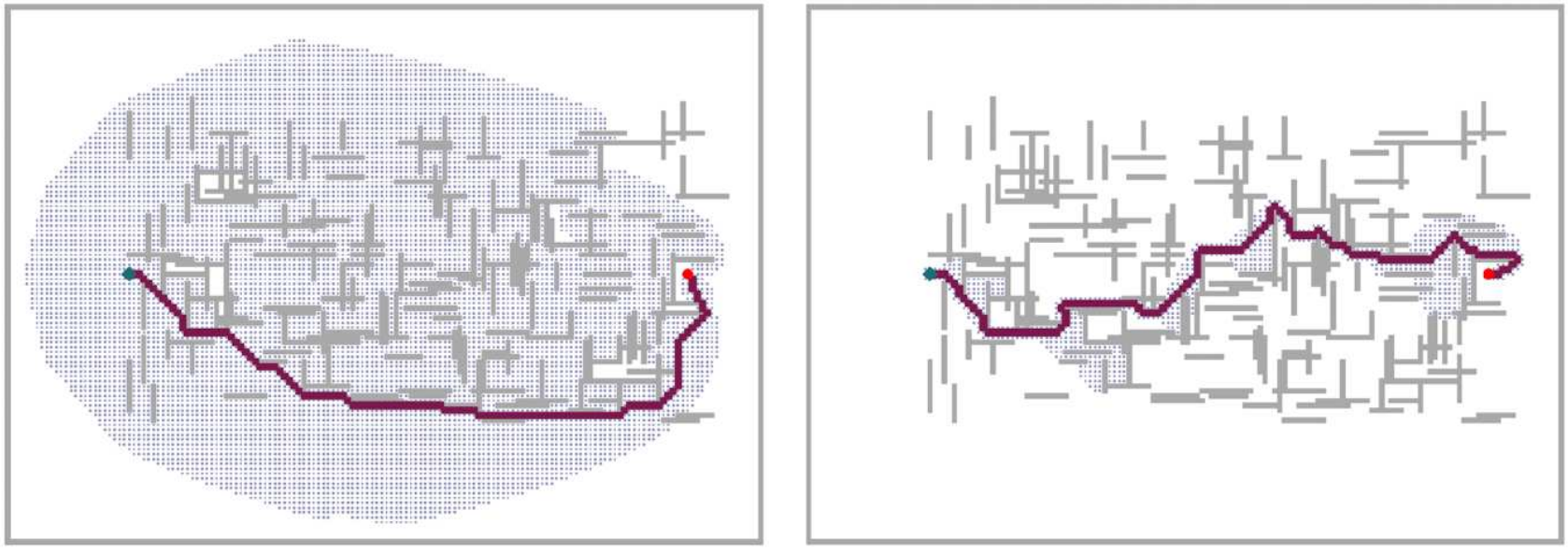
Fie problema Pacman cu scopul de a manca toate punctele din labirint. Fiecare actiune are un cost 1. Alegeti euristica/euristicile care sunt admisibile

- numarul total de puncte ramase
- distanta pana la cel mai apropiat punct
- distanta pana la cel mai indepartat punct
- distanta pana la cel mai apropiat punct plus distanta pana la cel mai indepartat punct
- nici una

- derivare din subprobleme - pattern database
- invatare din experienta

Obs: folosirea euristicilor inconsistente poate duce la solutii aproape optime cu cautare redusa

Weighted A* $f(n) = g(n) + w * h(n)$



(a) (b)

Two searches on the same grid: (a) an A* search and (b) a weighted A* search with weight $W = 2$. The gray bars are obstacles, the purple line is the path from the green start to red goal, and the small dots are states that were reached by each search. On this particular problem, weighted A* explores 7 times fewer states and finds a path that is 5% more costly.

Suboptimal

Bounded suboptimal search - costul solutiei e garantat $\leq w*$ cost optim
Bounded-cost search - cost solutie $\leq C$
Unbounded-cost search
vezi suboptimal

Rezumat

Formularea problemei necesita abstractizarea detaliilor lumii reale pentru a defini un spatiu de stari care poate fi explorat
Exista mai multe strategii de cautare neinformate
Iterative deepening search utilizeaza spatiu liniar si nu cu mult mai mult timp decat ceilalti algoritmi de cautare neinformata
Graph search poate fi exponential mai eficient decat tree search
A^* expandeaza nodurile cu $f(n) = g(n) + h(n)$ minim. E complet si optimal daca h(n) este admisibila, respectiv consistenta pt Tree-search, Graph-Search.

- see new algorithm for old problem
-
  Article - negative weight single source sortest paths in Near-linear Ti
-
  Using search and neural networks for prooving mathematical theorem

see State of AI report 2022